

QWEN

Helix Constitution — Universal QWEN.md

Field	Value
Revision	19
Created	2026-05-20
Last modified	2026-06-06T15:30:00Z
Status	active
Status summary	Added §11.4.127 mirror — Session-handoff resumption-prompt mandate (User mandate 2026-06-06): when a fresh session is needed (context limits/degradation) OR the operator asks whether a new session is needed, the agent MUST ALWAYS prepare + proactively provide a ready-to-paste resumption prompt valid for that EXACT moment + project phase (SHORT first-sentence + FULL block on demand) pointing to the live handoff docs (.remember/remember.md if present + docs/CONTINUATION.md §12.10, read FIRST + git fetch --all), stating PHASE + NEXT action + terminal goal, embedding exact live-state anchors (build IDs/MD5, device serials, HEAD, in-flight PIDs+logs, evidence paths) + binding constraints (anti-bluff §11.4, no-force-push §11.4.113, exact version/naming, hardware gotchas); MUST be moment-valid never generic; handoff docs current BEFORE the prompt; missing/stale/generic = §11.4.127 violation. Composes §12.10/.6/.66/.87/.103/.126. Classification: universal (§11.4.17). Prior round: Added §11.4.126 mirror — Default autonomous-loop working mode from first prompt (User mandate 2026-06-04): the endless fully-autonomous loop is the DEFAULT working mode, engaged automatically the moment the operator sends the FIRST request/prompt of a session — no per-session activation handshake; the loop continues until EITHER (A)

Field	Value
	a new fully-validated-and-verified version (tag) is created AND published across all owned submodules + main repo to all remotes (per §2.1/§11.4.40/§11.4.113), OR (B) for non-release main-stream work that work is fully completed AND all side work streams done AND nothing is left in the working queue for the current scope; STOPS ONLY on explicit operator STOP, empty current-scope queue, or §12 host-safety; goal ABSOLUTE EFFICIENCY; mimicking/imitation/false-results/bluff of ANY kind ABSOLUTELY FORBIDDEN (composes the §11.4 anti-bluff covenant — actually perform the work, never narrate/pretend it); §11.4.126 is the CAPSTONE promoting §11.4.87 from opt-in to always-on. Composes §11.4.87/.94/.97/.101/.103/.66/.6/.40/.42/.72/.113 + §2.1 + §12. Classification: universal (§11.4.17). Prior round: Added §11.4.125 mirror — Code-review-agent gate before pre-build + main build (mandatory multi-layer review) (User mandate 2026-06-04): after all batch fixes/changes/implementations are done and BEFORE the pre-build test sweep + main (artifact) build, the agent MUST dispatch dedicated code-review agent(s) (sub-agent-driven per §11.4.70/.20) that analyze all batch work + all existing data/facts/captured-evidence + the existing codebase (blast radius) + current git history, determine quality + safety + will-it-REALLY-work, and validate+verify every covering test genuinely exercises the work-under-test and catches its negation with ZERO false-result/bluff possibility; any finding (defect / error-prone change / safety risk / will-not-work / bluff-capable test / coverage gap) MUST be fixed+polished+improved+test-covered (four-layer §11.4.4(b), TDD-RED-first §11.4.43/.115) BEFORE the build proceeds; review iterates until no blocking findings; one of MULTIPLE STRONG LAYERS complementing (never replacing) the pre-build sweep + §11.4.92 multi-pass (author-side) + §11.4.108 + §11.4.110. Composes §11.4/.1/.4/.6/.40/.43/.50/.70/.20/.92/.102/.107/.108/.110. Classification: universal (§11.4.17). Prior round: Added §11.4.124 mirror — Dead/unwired-code investigate-before-remove mandate (User mandate 2026-06-04): "zero importers ⇒ dead ⇒ delete" is a guess (§11.4.6), never a finding — before removing ANY seemingly-dead element (zero-importer / never-called / unwired function/type/file/module/package/asset/config/build-target) the agent MUST FIRST investigate via git history (git log --follow, -S/-G pickaxe, blame on the deleted call-site) and capture as FACT where/how it was wired in + when/how it became dead + whether "no references" is

Field	Value
	real or a hidden reference (reflection/DI/build-tags/co-degen/plugin/FFI/config) the static tool cannot see; removal is permitted ONLY with captured PROOF it is genuinely unneeded AND MUST be its own separate descriptive commit citing the git-history evidence (+ §11.4.122 operator-confirmation for end-user capabilities; tracked via §11.4.90 Obsolete); with NO such proof the element MUST NOT be removed — instead wire it in properly (§11.4.114) and add any missing/unwired tests (§11.4.27/.43/.115); ALWAYS extra careful — when uncertain, default to NOT removing per §11.4.6/.101/.122. Classification: universal (§11.4.17). Prior round: Added §11.4.123 mirror — Rock-solid-proof-or-deep-research mandate (User mandate 2026-06-03): every reported issue / fix / claimed completion MUST be 100% validated with rock-solid CAPTURED proof (§11.4.5/.69/.107) before fixed/implemented/completed; metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden (§11.4/.1); when UNSURE how to validate, the agent MUST ALWAYS first perform deep web research (§11.4.8+§11.4.99) to discover/build an evidence-producing method — declaring "untestable" or accepting a metadata-only PASS without exhausting that research is itself a §11.4.123 violation; forensic case study (FACT) 1.1.8-dev FLAG_SECURE + non-introspectable-UI validation unclear → deep research yielded the CV/OCR/liveness/sink-probe oracle stack (§11.4.107/.112/.117). Classification: universal (§11.4.17). Prior round: Added §11.4.122 mirror — No-silent-removal-of-existing-components-without-operator-confirmation mandate (User mandate 2026-06-03): no application/component/service/package/feature/driver/module/prebuilt — any already-existing end-user capability — may be removed from the existing codebase/System without FIRST interactively asking the operator (§11.4.66, never silent/autonomous) + an EXPLICIT keep-or-remove decision; silent removal is a release blocker; forensic case study F2 Apple-TV-class app + F4 Huawei HMS component removed without asking, operator reversed both; tracked DROP path ask→approve→obsolete reason feature-removed+citation (§11.4.90)→remove. Classification: universal (§11.4.17). Prior round: Added §11.4.114-§11.4.121 mirrors (eight new universal anchors from the 1.1.8-dev live-defect remediation, 2026-06-03): §11.4.114 last-known-good-tag regression isolation; §11.4.115 RED-baseline-on-the-broken-artifact + polarity-switch (refines §11.4.43); §11.4.116 real-time conductor↔autonomous-test-framework sync channel (append-only event stream + atomically-rewritten

Field	Value
	status snapshot, verdict events carry evidence paths); §11.4.117 CV/OCR pixel-oracle fallback for non-introspectable UIs (refines §11.4.48/.52/.107); §11.4.118 discovery-pressure to confirm known-issue-set completeness; §11.4.119 single-resource-owner partitioning for parallel hardware testing (refines §11.4.58/.103); §11.4.120 fix-breaks-its-own-gate reconciliation (rewrite the gate, never fake-pass / never revert the fix); §11.4.121 no-commit-while-build-writes-tracked-artifacts (build-output analogue of §11.4.84). All Classification: universal (§11.4.17). §11.4.108–§11.4.113 + §11.4.78–§11.4.107 mirrors continue from earlier Revisions.
Issues	none
Issues summary	—
Fixed	§11.4.108 mirror
Fixed summary	§11.4.107 lands in lockstep with the Constitution.md §11.4.107 addition.
Continuation	—

Table of contents

- [How inheritance works](#)
- [Identity & posture](#)
- [Top-level invariants for every agent](#)
- [Critical base rules restated \(for agents that don't honour @imports\)](#)
 - [§11.4 — Anti-bluff covenant — END-USER QUALITY GUARANTEE](#)
 - [§1.1 — Mutation-paired gates](#)
 - [§11.4.10 — Credentials-handling mandate](#)
 - [§11.4.17 — Universal-vs-project classification](#)
 - [§11.4.20 — Subagent-driven-by-default](#)
 - [§11.4.73 — Main-specification document versioning + revision discipline](#)
 - [§11.4.74 — Submodule-catalogue-first discovery + extend-don't-reimplement](#)
 - [§11.4.76 — Containers-submodule mandate](#)
- [Companion documents](#)

How inheritance works

This QWEN.md is for the **Qwen Code CLI agent** (qwen-code). It documents the universal Helix Constitution from the Qwen agent's perspective — the same content seen by Claude Code via CLAUDE.md and by OpenCode / Cursor / generic AI tooling via AGENTS.md.

The **authoritative source** is Constitution.md in this repository. When this file diverges from Constitution.md, the latter wins. This file exists because:

1. Qwen Code does not always honor @import directives across files; restating the critical rules inline ensures the agent reads them on every session.
2. Cross-agent parity: Claude Code (CLAUDE.md), OpenCode/ Cursor (AGENTS.md), Qwen Code (QWEN.md) all start from the same constitutional baseline.

Discover this file via the canonical parent-walk pattern documented in every consuming project's CLAUDE.md / AGENTS.md:

```
find_constitution_root() {  
    local cur="${1:-$PWD}"  
    while [[ "${cur}" != "/" ]]; do  
        if [[ -f "${cur}/constitution/Constitution.md" ]]; then  
            echo "${cur}/constitution"; return 0  
        fi  
        cur="$(dirname "${cur}")"  
    done  
    return 1  
}
```

Identity & posture

When the Qwen Code agent loads this file as part of session bootstrap, it operates under the Helix Constitution with the same identity, anti-bluff posture, and end-user quality covenant as any other CLI agent in the catalogue. There is no "Qwen-lite" interpretation — Qwen Code is held to the full §11.4 covenant.

Top-level invariants for every agent

1. **Read order on a cold start.** CLAUDE.md / AGENTS.md / QWEN.md (this file) for the AI-agent posture; Constitution.md for the canonical universal rules; then the consuming project's CLAUDE.md for project-specific extensions.
2. **Multi-mirror push.** Every commit on this submodule MUST land on all four canonical mirrors (GitHub + GitLab + GitFlic + GitVerse) per §2.1.

3. **Anti-bluff is non-negotiable.** Passing tests MUST imply working features. Bluffing PASSes (and FAILs that hide root cause) is a release blocker (§11.4 + §11.4.1).
4. **Submodule-catalogue-first.** Before scaffolding anything, survey the vasic-digital + HelixDevelopment orgs for an existing Submodule (§11.4.74). Reuse → extend → no-match in that order.
5. **Containers-submodule for ALL container workloads.** Use vasic-digital/containers (§11.4.76) — never reinvent compose/boot orchestration in-project.

Critical base rules restated (for agents that don't honour @imports)

§11.4 — Anti-bluff covenant — END-USER QUALITY GUARANTEE

Forensic anchor — verbatim user mandate (2026-04-28, reasserted 2026-05-21, 2026-05-29):

"all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

Operative rule. Captured tests MUST exercise the behavior they claim to verify. PASS-without-execution paths, mock-only tests that don't round-trip, skip-by-default integration tests that mask real failures, metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-without-runtime PASS are all critical defects regardless of how green the summary line looks. Each test failure in CI MUST imply a real broken feature; each PASS MUST imply the feature actually works for the end user. Tests and HelixQA Challenges are bound EQUALLY.

The verbatim covenant MUST be present in every consumer governance file (project Constitution.md, CLAUDE.md, AGENTS.md, QWEN.md) and in every owned submodule's equivalent. Tools that don't expand @imports still read the literal text.

§1.1 — Mutation-paired gates

Every captured-evidence gate **MUST** ship with a paired mutation test that mutates the gate's anchor + runs the gate + asserts the gate now **FAILs**. Absence of the paired mutation is itself a bluff.

§11.4.10 — Credentials-handling mandate

Credentials are **NEVER** committed to git, **NEVER** logged, **NEVER** printed to stdout/stderr. `.env` files are `.gitignored`; committed siblings are `.env.example` placeholders only. Pre-store leak audit (§11.4.10.A) scans every PR for credential patterns before merge.

§11.4.17 — Universal-vs-project classification

Every new rule **MUST** declare itself **universal** (applies to every project consuming this Constitution) or **project** (applies only to one specific project). Misclassification (universal rule landing in a project's local `CLAUDE.md` instead of the constitution submodule) is a release blocker.

§11.4.20 — Subagent-driven-by-default

When a CLI agent (Qwen Code included) has a subagent / Task tool available, the default mode of operation is to dispatch sub-agents for discrete units of work rather than perform them inline. This protects context, enables parallel work, and matches the §11.4.27 no-fakes-beyond-unit-tests posture (each subagent runs real tools).

§11.4.73 — Main-specification document versioning + revision discipline

Projects with a main specification (`docs/specs/.../specification.V<N>.md`-shaped) maintain **TWO** version axes:

- **Primary** (V1 / V2 / V3 ...) bumps for major rewrites only.
- **Secondary** (Revision N in the metadata table) bumps for additive changes within a primary version.

Spec edits trigger mandatory comprehensive planning and implementation in the consuming project — they are not isolated doc tweaks (the "spec ripple" rule).

§11.4.74 — Submodule-catalogue-first discovery + extend-don't-reimplement

Direct user mandate (verbatim, 2026-05-20): "We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in vasic-digital and HelixDevelopment organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times. For any missing features we MUST IMPLEMENT them properly and extend those Submodules further!"

Before scaffolding any new module / package / helper / utility, the Qwen Code agent MUST: (1) survey vasic-digital + HelixDevelopment on GitHub + GitLab — the canonical inventory is submodules-catalogue.md (142 repos categorised); (2) reuse an existing Submodule when it covers $\geq 80\%$; (3) extend in-place via upstream PR when $80\%+$ matches; (4) document the survey result via Catalogue-Check: `reuse|extend|no-match <org/repo>@<sha>` in the tracker row.

Every Submodule in the catalogue is subject to the same development-cycle rules (§11.4 anti-bluff, §1.1 paired mutations, §11.4.10 credentials, §11.4.61 metadata + ToC, §11.4.65 universal export, §11.4.73 spec versioning, §2.1 multi-mirror push, §3 propagation order).

§11.4.76 — Containers-submodule mandate

Direct user mandate (verbatim, 2026-05-20): "For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemu / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly: <https://github.com/vasic-digital/containers> (`git@github.com:vasic-digital/containers.git`). Containers Submodule contains all means for us to Containerize our code and services! If any feature or Containerizing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! No bluff work is allowed of any kind!"

For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / container-backed emulators), the Qwen Code agent MUST: (1) install `vasic-digital/containers` (`digital.vasic.containers`) as a Git submodule when scaffolding the consuming project; (2) consume via `replace` directive during development + pinned commit SHAs in production; (3) boot infra on-demand via the Submodule's `pkg/boot` + `pkg/compose` + `pkg/health` APIs so the operator is never required to start `podman` machine / `docker compose` up manually — the boot is part of the test entry point (**on-demand-infra invariant**); (4) extend the Submodule via upstream PR when a runtime / lifecycle primitive is missing —

never reimplement in-project (per §11.4.74); (5) anti-bluff: integration tests claiming to exercise containerized components **MUST** actually boot them via the Submodule. Short-circuit fakes that bypass boot are a §11.4 violation. A passing test **MUST** imply the infra was up.

Tracker rows touching containerization **MUST** record Catalogue-Check: extend vasic-digital/containers@<sha> (or reuse); no-match requires demonstrating the Submodule cannot model the workload.

Planned anti-bluff gate CM-CONTAINERS-USED scans container-touching PRs for digital.vasic.containers/... imports. Paired mutation strips the import + asserts FAIL.

Canonical authority: Constitution.md §11.4.76.

Non-compliance: reinventing compose orchestration in-project is a release blocker.

§11.4.77 — Regeneration-mechanism-required mandate

Direct user mandate (verbatim, 2026-05-20): "We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content! Add this mandatory safety rule / constraint into ours root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md, QWEN.md or any other required document / file from the constitution Submodule!"

Forensic anchor. 2026-05-20T15:00Z: ATMOSphere parent's audio Tier 1 commit_all.sh stalled 4 h on git add -A scanning 274 GiB .git-backup-* + 159 GiB RKTools/linux/ + 167 GiB qa-results/ — all untracked but un-gitignored. Bare .gitignore fix would orphan every fresh clone (missing RKTools, missing test infra, missing build outputs).

Every .gitignore entry the Qwen Code agent considers adding that excludes (a) >~100 MiB OR (b) any artefact essential to building / running / testing the project **MUST** carry a documented + automated mechanism to either **re-obtain** (vendor tarball, SDK installer, package registry, git submodule, object store) OR **re-generate** (build pipeline, code-gen, asset render, captured-evidence replay, container build).

Required artefacts per qualifying .gitignore entry: (1) .gitignore-meta/<entry-slug>.yaml declaring pattern + mechanism-type + script-path + expected-disk-usage + vendor-url-or-source + integrity hash + requires-network + requires-credentials; (2) post-clone bootstrap entry (scripts/setup.sh or canonical equivalent) running the mechanism non-interactively; (3) pre-build gate

verifying regenerated content present OR stamp .gitignore-meta/.regenerated/<slug>.ok recent; (4) README + docs/guides/*.md describing the mechanism + manual fallback + time/disk budget + per-§11.4.10 credentials.

No escape hatch: bare .gitignore additions without the mechanism are §11.4 PASS-bluff variants — codebase appears complete but fresh clone cannot build / run. No --skip-regen-mechanism, --gitignore-is-enough, --operator-already-has-content flag.

Planned anti-bluff gate CM-GITIGNORE-REGEN-MECHANISM scans every .gitignore addition for matching .gitignore-meta/ sibling. Paired §1.1 mutation strips a required YAML key → gate FAILs.

Composes with §11.4.6, §11.4.65, §11.4.66, §11.4.71, §11.4.74, §11.4.75, §11.4.76, §9 / §9.2, §3.

Canonical authority: Constitution.md §11.4.77.

Non-compliance is a release blocker regardless of context.

§11.4.78 — CodeGraph code-intelligence mandate

Direct user mandate (verbatim, 2026-05-20): "Make codegraph MANDATORY CHOICE for this purpose for all of our project ... All project which do not have configured and installed codegraph yet MUST DO IT and MUST USE IT!"

Every consuming project worked on by AI coding agents — Qwen Code included — MUST install, initialize, and use **CodeGraph** (<https://github.com/colbymchenry/codegraph>, npm @colbymchenry/codegraph): a local SQLite semantic code-knowledge-graph exposed to agents over MCP, 100% local. Install globally via npm (no sudo). codegraph init + codegraph index: .codegraph/config.json is tracked, .codegraph/codegraph.db is gitignored with codegraph index as its §11.4.77 regeneration mechanism; the exclude list MUST exclude other-owned submodules and every §11.4.10 credential/secret path. Wire the codegraph serve --mcp server into every CLI agent the developers use — for Qwen Code via .qwen/settings.json (qwen mcp add codegraph codegraph serve --mcp --scope project --transport stdio). Cover the integration with an anti-bluff verification suite using an unforgeable per-agent challenge (a fact obtainable only by calling a CodeGraph MCP tool); un-runnable agents are documented SKIP gaps, never faked PASSes. Document in docs/CODEGRAPH.md. CodeGraph is consumed as the npm package (§11.4.74), not a git submodule.

Composes with §11.4.3, §11.4.10, §11.4.12, §11.4.65, §11.4.30, §11.4.74, §11.4.77, §11.4, §1.1.

Canonical authority: Constitution.md §11.4.78.

Non-compliance is a process violation; a project worked on by AI agents without CodeGraph installed, wired, and anti-bluff-verified is in breach.

§11.4.79 — Own-org submodules MUST be included in the CodeGraph index

Direct user mandate (verbatim, 2026-05-21): "All Submodules we use in the project and that are part of organizations to which we have the full access via GitHub, GitLab and other CLIs MUST BE included into the codegraph database and initialized / scanned / synced!"

Refines §11.4.78 step 2 with a per-submodule-ownership split. **Own-org submodules** — full write access via the project's CLIs (canonical orgs: vasic-digital + HelixDevelopment) — MUST be INCLUDED in the index so Qwen Code (and every other AI agent) sees a unified call-graph across the project's domain + own-org infra code. **Third-party submodules** (the §11.4.74 no-match → vendor path) MUST be EXCLUDED. Operational steps: (1) git submodule update --remote --merge to pull latest before re-indexing — respect load-bearing third-party pins. (2) Adjust .codegraph/config.json exclude list. (3) Re-index via scripts/codegraph_setup.sh. (4) Validate via scripts/codegraph_validate.sh probe resolving a symbol that lives ONLY inside an own-org submodule. (5) Paired §1.1 mutation: temporarily add own-org path to exclude → validate FAILs → restore.

Composes with §11.4.74, §11.4.78, §11.4.10, §1.1, §107.

Canonical authority: [Constitution.md](#) §11.4.79.

Non-compliance is a process violation; severe cases (own-org submodules silently excluded without an audit trail in .codegraph/config.json) are release blockers.

§11.4.80 — CodeGraph regular-update + sync automation mandate

Direct user mandate (verbatim, 2026-05-21): "We MUST regularly check for the updates and execute codegraph npm updates so the latest version of it is always installed [...] Make sure we have proper full automation bash scripts which will run regularly and that these are part of the constitution Submodule [...] Make sure all updates, sync processes we do and important codegraph related events are all documented under docs/codegraph in Status and Status_Summary documents [...]"

Three deliverables in this constitution submodule: (1) scripts/codegraph_update.sh — npm-installs latest; §107 anti-bluff verifies codegraph --version reflects the new version. (2) scripts/

codegraph_sync.sh — runs status → sync → status → project's validate; appends to both project's + constitution's docs/codegraph/Status.md. (3) docs/codegraph/Status.md + Status_Summary.md ledgers, exported per §11.4.65 to .html + .pdf.

Cadence: weekly floor (§11.4.45). Scripts inherited by reference (§3) — Qwen Code consuming projects invoke them at \${CONST_DIR}/scripts/codegraph_*.sh, never copy.

Composes with §11.4.78, §11.4.79, §11.4.10, §11.4.45, §11.4.65, §107, §1.1.

Canonical authority: Constitution.md §11.4.80.

Non-compliance is a process violation; severe cases (>2 weeks since last update + open AI-agent work) are release blockers.

§11.4.81 — Cross-platform-parity mandate (User mandate, 2026-05-21)

Direct user mandate (verbatim, 2026-05-21): "Any Linux-only blocker / issue we have MUST BE created macOS and other supported platforms equivalent! So, depending on platform proper implementation will be used for particular OS! EVERYTHING MUST BE PROPERLY EXTENDED AND UPDATED!"

Every consuming project whose supported-platforms manifest lists more than one OS MUST ship per-OS-equivalent implementations + tests for every feature/gate/challenge/mutation that depends on platform-specific primitives. Runtime dispatch via `uname -s` (or equivalent platform detection). Three sub-mandates: **(A)** Per-OS implementation REQUIRED — `cgroup/systemd//proc` primitives have documented equivalents (POSIX `setrlimit/ulimit`, `launchd`, `rctl`, Job Object). **(B)** Per-OS tests REQUIRED — every gate test has case "`$(uname -s)`" in branches with positive captured evidence per §11.4.2 + §11.4.5 in each branch; SKIP-with-reason only when the platform genuinely cannot enforce. **(C)** Honest kernel-gap citation + adjacent equivalent test REQUIRED where no equivalent exists (canonical: XNU does NOT enforce `RLIMIT_AS` for unprivileged processes → SKIP with exact reproducer + adjacent test of what IS enforced, e.g. `RLIMIT_CPU+SIGXCPU`). The adjacent test is itself anti-bluff per §11.4 with a paired §1.1 mutation.

Per-OS equivalence catalogue (canonical): `systemd-run --user --scope ↔ POSIX ulimit -t -u / launchd; cgroup MemoryMax ↔ XNU gap (use RLIMIT_CPU adjacent) / rctl / Job Object; cgroup TasksMax ↔ RLIMIT_NPROC; /proc/<pid>/oom_score_adj ↔ no Darwin/BSD equivalent.`

Composes with §11.4.1 / §11.4.2 / §11.4.3 (strictened — SKIP only when kernel cannot) / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.20 / §11.4.27 / §11.4.69 / §11.4.70 / §107. Pre-build gate CM-CROSS-PLATFORM-PARITY + paired §1.1 mutation. No escape hatch.

Canonical authority: Constitution.md §11.4.81.

Non-compliance is a release blocker on multi-platform projects.

§11.4.82 — Iteration-speedup discipline mandate (User mandate, 2026-05-22)

Direct user mandate (verbatim, 2026-05-22): "How can we speed-up this whole development and fixing process? ... all speed optimizations critical rules and mandatory constraints MUST BE all added into our root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md and QWEN.md!"

Iteration cycle time bounds the project's defect-discovery rate. Slow cycles ship fewer validated features per unit of operator time. The 2026-05-22 forensic session witnessed ~3 hours of operator wait on a job whose intrinsic compute was ~90 min — every wasted minute came from a missing speedup discipline.

Every consuming project's build / test / commit / debug pipeline MUST adopt the following 9 speedup disciplines AS MANDATORY:

- **(A) Phase 1 forensic before any speculative source patch.** Speculative patches without root-cause evidence are §11.4.6 + §11.4.82 violations.
- **(B) Live-ADB-First (or live-equivalent) before any re-build.** §11.4.51 strengthened to release-blocker. Skipping live-probe for LIVE_ADB_TESTABLE changes = §11.4.82 violation.
- **(C) Pre-flight before rebuild orchestrators.** 30-sec readiness check verifies device + sink reachability, memory budget, lock files, orphan processes.
- **(D) Persistent build caches outside containers.** ccache + Soong / sccache / Gradle daemon state bind-mounted to host. Subsequent rebuilds drop from ~40 min to 5-15 min.
- **(E) Module-only rebuild for CONFIG_*=m driver patches.** make modules on single driver dir saves 15-20 min/cycle when applicable.
- **(F) Parallel multi-device testing.** Every owned validation device runs the autonomous cycle concurrently with separate output dirs. Catches per-device defects one cycle earlier.
- **(G) Subagent scope ≤30 min + worktree isolation + single-responsibility.** Empirical pattern: 8-task subagents stall, 2-3-task subagents complete.
- **(H) Lock-file + stale-process hygiene.** Clean .git/index.lock + orphan processes on session start. Disable auto git-gc in concurrent multi-agent repos.

- **(I) Cycle telemetry per §11.4.24.** Every cycle logs commit, per-phase wall-clock, speedup flag set, outcome. Weekly aggregation surfaces empirical ROI per discipline.

Composes with §11.4.4 / §11.4.6 / §11.4.9 / §11.4.20 / §11.4.24 / §11.4.42 / §11.4.43 / §11.4.50 / §11.4.51 / §11.4.52 / §11.4.58 / §11.4.70 / §12.7 / §107. Pre-build gate CM-ITERATION-SPEEDUP-DISCIPLINE (when implemented) + paired §1.1 mutation. No escape hatch — no --skip-phase1, --no-pre-flight, --rebuild-everything, --unlimited-subagent-scope, --ignore-locks, --no-telemetry flag exists.

Canonical authority: [Constitution.md](#) §11.4.82.

Non-compliance is a release blocker.

§11.4.83 — docs/qa/ end-user evidence mandate (User mandate, 2026-05-22)

Direct user mandate (verbatim, 2026-05-22): "every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under docs/qa/<run-id>/ (per-feature subdirectories). A feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation MUST preserve full bidirectional communication threads as proof."

Every feature that ships MUST carry a recorded end-to-end communication transcript plus attached materials committed under docs/qa/<run-id>/. Transcripts MUST be full bidirectional. Attached materials live in-repo (no external-only links — §11.4.13 sink-side violation). Bot-driven / agent-driven QA MUST preserve the full conversation thread as the proof artefact. CI release gates MUST refuse to tag a version whose feature-shipping commit lacks its docs/qa/<run-id>/.

Composes with §11.4.2, §11.4.5, §11.4.13, §11.4.65, §11.4.69, §107, §1.1.

Canonical authority: [Constitution.md](#) §11.4.83.

Non-compliance is a release blocker.

§11.4.84 — Working-tree quiescence rule for subagent commits (User mandate, 2026-05-22)

Short tag: working-tree quiescence.

Direct user mandate (verbatim, 2026-05-22): "no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before git add, the committing agent MUST grep

its own working tree for mutation markers (MUTATED for paired, // always pass, return json.Marshal shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT."

No subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Pre-git add: grep for mutation markers (MUTATED for paired, // always pass, return json.Marshal shortcuts, // MUTATION annotations, _mutated_* filenames). Cross-check git status --porcelain against declared scope → unaccounted entries ABORT.

Lesson (forensic). A consuming project's logo-fix subagent (Herald commit 72e81ab, 2026-05-21) swept a // always pass JWT-bypass mutation residue into an unrelated commit, pushed to all four mirrors before being caught. Fix d5bd360 landed within the hour, but the security-defect window is real and the lesson is constitutional.

Operative rule. (1) Pre-git add grep + scope-match → unaccounted ABORT. (2) Active mutation gates MUST be serialised before unrelated commits. (3) Concurrent subagents in same checkout MUST use lockfile (.git/MUTATION_IN_PROGRESS) or git worktree add. (4) Pre-push mutation-residue-scanner BLOCKS pushes containing mutation markers.

Composes with §1.1, §11.4.20, §11.4.70, §11.4.27, §11.4.10, §11.4.71, §107.

Canonical authority: [Constitution.md](#) §11.4.84.

Non-compliance is a release blocker.

§11.4.85 — Stress + Chaos Test Mandate (User mandate, 2026-05-24)

Short tag: stress-chaos-mandate.

Direct user mandate (verbatim, 2026-05-24): "Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!"

Every fix MUST ship with full-automation stress + chaos test suites. Happy-path-only coverage is a §11.4 / §107 PASS-bluff at the resilience layer.

Stress = sustained load ($N \geq 100$ iters OR ≥ 30 s) + concurrent contention ($N \geq 10$ parallel) + boundary conditions (empty/max/off-by-one). **Chaos** = failure-injection per §11.4.69 closed-set (process-kill, network-drop, input-corruption, OOM, disk-full, state-corruption) + verifiable recovery.

Anti-bluff: every stress + chaos PASS cites a captured-evidence artefact (latency.json / categorised_errors.txt / state_delta_snapshot.json) per §11.4.5 + §11.4.69. Helper library stress_chaos.sh provides ab_stress_run / ab_stress_concurrent / ab_chaos_* primitives. Chaos cleanup non-negotiable — corrupt-re-store / disk-fill-cleanup / process-restart MUST run in trap EXIT.

4-layer per §11.4.4(b): pre-build gate (test files exist + parse) + paired meta-test mutation + on-device test (if LIVE_ADB_TEST-ABLE) + HelixQA Challenge entry. Composes with §11.4 / §11.4.1 / §11.4.5 / §11.4.6 / §11.4.43 / §11.4.50 / §11.4.52 / §11.4.69 / §11.4.83.

Canonical authority: [Constitution.md](#) §11.4.85.

Non-compliance is a release blocker. No --skip-stress, --no-chaos, --happy-path-suffices flag exists.

§11.4.86 — Roster/corpus-backed Status-doc auto-sync mandate (User mandate, 2026-05-25)

Forensic anchor (verbatim): "Make sure that assets and players Status docs are ALWAYS regularly updated and in sync like all others Status docs — any time we add or modify the assets content(s) or we change or add new / remove existing pre-installed video and audio player apps! This MUST WORK OUT OF THE BOX!"

Status docs (§11.4.45) backed by a **tracked roster** (installed apps/components) or **asset corpus** (test/media directory) MUST resync out of the box the moment a member changes — Status doc + Status_Summary + HTML + PDF, mechanically. Mechanism (all must hold): (1) drift-proof **fingerprint** = sha256 of sorted member list (NOT mtime), persisted in a sidecar; (2) **sync helper** regenerates fingerprint + re-exports (via §11.4.65); (3) **pre-build gate** FAILs on fingerprint drift (mirrors §11.4.12 + §11.4.45); (4) paired §1.1 mutation. Composes with §11.4.12 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §11.4.60 / §11.4.65 / §11.4.6. Universal (§11.4.17) — consuming project supplies the specific docs/roster/helper/gate per §11.4.35.

Canonical authority: [Constitution.md](#) §11.4.86.

Non-compliance is a release blocker. No --skip-roster-sync, --allow-status-drift flag exists.

§11.4.87 — Endless-loop autonomous work + zero-idle agent dispatch + anti-bluff testing mandate (User mandate, 2026-05-26)

When operator instructs an AI agent to "continue in endless loop fully autonomously" (or semantically-equivalent), the agent MUST treat as HARD-CONTRACT covenant: (A) continue until docs/Issues.md non-terminal Status entries = 0 AND docs/CONTINUATION.md §3 Active work empty AND no subagent in-flight AND no external dep in-flight; (B) dispatch background subagents for parallelisable work — main + subagents concurrent, "waiting for results" is the ONLY idle reason; (C) every closure lands four-layer test coverage per §11.4.4(b) with captured-evidence "physical proofs" (audio/video/network/UI/sysfs) — metadata-only / config-only / absence-of-error / grep-without-runtime PASS are critical defects; (D) §11.4 anti-bluff covenant family operative end-to-end (tests AND HelixQA Challenges bound equally per forensic anchor "tests pass but features don't work"); (E) loop terminates ONLY on all-conditions-met, explicit operator STOP, host-safety demand, or scheduled wake on known-future-actionable signal.

Composes with §11.4 / §11.4.1 / §11.4.2 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.7 / §11.4.20 / §11.4.27 / §11.4.42 / §11.4.43 / §11.4.50 / §11.4.52 / §11.4.58 / §11.4.68 / §11.4.69 / §11.4.70 / §11.4.83 / §11.4.85 / §11.4.86 / §12.10. Pre-build gate CM-COVENANT-114-87-PROPAGATION + paired §1.1 mutation.

Canonical authority: Constitution.md §11.4.87.

Non-compliance is a release blocker. No --idle-OK, --skip-endless-loop, --bluff-permitted-for-this-task, --metadata-only-test-suffices, --no-physical-proof-required flag exists.

Companion documents

- Constitution.md — authoritative universal Constitution. ALWAYS the tie-breaker.
- CLAUDE.md — Claude Code agent's view of the universal constitution.
- AGENTS.md — OpenCode / Cursor / generic-tooling view.
- README.md — high-level overview + multi-mirror contract.

When operating in a consuming project, ALSO read the project's local QWEN.md / CLAUDE.md / AGENTS.md for project-specific extensions; these MUST NOT weaken any universal rule but MAY add stricter project-specific constraints.

§11.4.88 — Background-push mandate: commit-lock release immediately after commit, push runs detached (User mandate, 2026-05-26)

Forensic anchor: 2026-05-26 `commit_all.sh` held flock ~5h on synchronous post-commit push. Mandate: (A) flock release IMMEDIATELY after commit; (B) push detached via `nohup` + `disown`; (C) `push_all.sh` per-remote flock — same-remote serializes, different-remote parallel; (D) failures → `qa-results/push_failures/`, autonomous loop checks per §11.4.87(A); (E) `--sync-push` escape for §11.4.41 force-push only. Composes §2.1 / §9.2 / §11.4.41 / §11.4.42 / §11.4.71 / §11.4.87. Gates CM-COVENANT-114-88-PROPAGATION + CM-BACKGROUND-PUSH-WIRED + paired §1.1 mutations.

Canonical authority: [Constitution.md](#) §11.4.88. Non-compliance is a release blocker.

§11.4.89 — Background test execution mandate (User mandate, 2026-05-27)

Forensic anchor 2026-05-27: conductor invoked long `pre_build` synchronously, blocking main stream 6-7 min on §JV/§JW/§JX/§JY scaffolding. Symmetric to §11.4.88 at test-execution layer. Mandate: (A) long tests (>30s) run via `nohup ... > <log> 2>&1 & + disown`; (B) main stream proceeds to §11.4.42 priority queue immediately; (C) hard-dependency gating via `poll/pgrep` — surface §11.4.66 options if still running; (D) failures land in log files; (E) foreground only <30s OR operator authorisation; (F) per-script flock. Composes §11.4.42 / §11.4.66 / §11.4.82 / §11.4.84 / §11.4.85 / §11.4.87 / §11.4.88. Gates CM-COVENANT-114-89-PROPAGATION + CM-BACKGROUND-TEST-EXECUTION-WIRED + paired §1.1 mutations.

Canonical authority: [Constitution.md](#) §11.4.89. Non-compliance is a release blocker.

§11.4.90 — Obsolete status + obsolescence audit (User mandate, 2026-05-27)

§11.4.15 Status closed-set + terminal `Obsolete` (→ `Fixed.md`). Reasons: `superseded-by-design-change` | `superseded-by-later-mandate` | `feature-removed` | `duplicate-of` | `unsupported-topology`. `**Obsolete-Details:**` line mandatory (Since + Reason + Superseding-item + Triple-check evidence). Colorizer `cell-status-obsolete` = light-gray + strikethrough. Audit cadence per §11.4.40 + §11.4.42. Triple-check non-negotiable.

Canonical authority: [Constitution.md](#) §11.4.90. Release blocker.

§11.4.91 — Summary-doc clarity (User mandate, 2026-05-27)

Every summary entry's description: self-contained, meaningful, ≥ 6 words / ≥ 40 chars, SUBJECT + PROBLEM/GOAL. Anti-patterns forbidden: section labels, bare metadata, §-letter alone. Generators extract from H1/H2 heading. Pre-build gate CM-SUMMARY-CLARITY-DESCRIPTIONS.

Canonical authority: [Constitution.md](#) §11.4.91. Release blocker.

§11.4.92 — Multi-pass change-evaluation (User mandate, 2026-05-27)

5-pass evaluation BEFORE commit-ready: (1) Main-task verification; (2) Regression-blast-radius; (3) Cross-feature interaction; (4) Deep-research per §11.4.8; (5) Anti-bluff confirmation. Doc per pass. Trivial exemption: typo/revision/MD-export ONLY if zero source touched.

Canonical authority: [Constitution.md](#) §11.4.92. Release blocker.

§11.4.93 — SQLite-SSoT for workable items (User mandate, 2026-05-27)

docs/.workable_items.db SQLite — DB authoritative, MD derived. Schema: items + item_history + obsolete_details + operator_block_details + firebase_metadata + meta. Go binary cmd/workable-items/ sync md↔db + diff + validate + add + close. Bidirectional regen byte-identical. Anti-bluff: unit+integration+stress+chaos per §11.4.85. Go binary in constitution submodule per §11.4.74. 6-phase migration \$LA.

Canonical authority: [Constitution.md](#) §11.4.93. Release blocker.

§11.4.94 — Zero-idle priority-first parallel-by-default (User mandate, 2026-05-27)

Binds §11.4.20/42/58/70/72/82/87/88/89 — always-on. Idle ONLY: externally blocked, operator STOP, §12 host-safety. Before any wake/sleep survey parallel-work queue. Priority MANDATORY (§11.4.72 audio first). Subagent-driven default non-trivial. Background default >30s. Stability per §11.4.92 + §11.4.84 + §12.6-9. Updates at milestones.

Canonical authority: [Constitution.md](#) §11.4.94. Release blocker.

§11.4.95 — Workable-items DB TRACKED in git (User mandate, 2026-05-27)

§11.4.93 amendment: docs/workable_items.db TRACKED, NEVER gitignored. Authoritative source. Every mutation → stage+commit+push DB + MD. WAL-checkpoint before commit. §11.4.77 does NOT apply. Pre-build gates CM-COVENANT-114-95-PROPAGATION + CM-WORKABLE-ITEMS-DB-TRACKED.

Canonical authority: [Constitution.md](#) §11.4.95. Release blocker.

§11.4.96 — Safe-parallel-work-with-long-build (User mandate, 2026-05-27)

SAFE during AOSP build: (A) docs/MD; (B) scripts/; (C) pre-build gates; (D) on-device tests; (E) constitution edits+push; (F) submodule push per §11.4.88; (G) read-only ADB probes; (H) sub-agent dispatch; (I) web research; (J) workable-items DB ops; (K) pre-build + meta-test execution backgrounded.

UNSAFE: (α) git checkout/reset on source; (β) mass deletes/re-names under built source; (γ) APK-built submodule pointer; (δ) out/; (ε) make clean; (ζ) container destruction; (η) disk-fill; (θ) §12 host-safety.

Conductor dispatches ≥ 1 (A)-(K) per pause. "Build running, nothing else to do" NEVER true.

Canonical authority: [Constitution.md](#) §11.4.96. Release blocker.

§11.4.97 — Max-use-of-idle + progress-update cadence (User mandate, 2026-05-27)

(A) Every minute progressable idle = §11.4.97 violation, dispatch CONTINUOUSLY; (B) 1-line operator updates at commit/subagent/anchor/evidence/migration — no prompt; (C) continuous physical-proof gathering per §11.4.5/6/69; (D) composes with operating-mode anchor family; (E) idle-only-when-blocked closed-set unchanged from §11.4.94(A). Pre-build gates CM-COVENANT-114-97-PROPAGATION + CM-IDLE-TIME-AUDIT.

Canonical authority: [Constitution.md](#) §11.4.97. Release blocker.

§11.4.98 — Full-Automation Anti-Bluff (User mandate, 2026-05-28)

Forensic anchor: "Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! ... No bluff is allowed!"

Closes the manual-intervention gap §11.4+85+87+89+94 didn't forbid. (A) every test MUST self-drive end-to-end — PASS/FAIL/SKIP-with-reason without further human action; (B) single exception = one-time credential bootstrap OUTSIDE test execution (.env, ~/.bashrc, OAuth, MTProto-session-activation); (C) concrete live requirements: (1) no "operator MUST type" prompts — drive via MTProto/real-user-API/webhook-fixture/in-process loopback; (2) no UUIDs colliding with active dev session (Herald 2026-05-28: silent exit -1 lesson); (3) no 60s human-response windows (§11.4.50 violation); (4) re-runnability -count=3 with self-cleaning state; (5) audit COMPLIANT vs NON-COMPLIANT; (6) no false-positive PASS — silent-skip-as-PASS + stale-evidence forbidden, §11.4.3 SKIP-with-reason correct; (D) composes §11.4.85+89+87+94 = continuously-validated fully-automated non-flake anti-bluff regime; (E) inheritance per §11.4.35 — restate literal 11.4.98 in every consumer; pre-build gate CM-COVENANT-114-98-PROPAGATION; paired §1.1 mutations strip → FAIL; (F) enforcement: manual-action commit BLOCKED; NON-COMPLIANT after 30 days → §11.4.90 Obsolete citing §11.4.98.

Canonical authority: [Constitution.md](#) §11.4.98. Release blocker.

§11.4.99 — Latest-Source Documentation Cross-Reference (User mandate, 2026-05-28)

Forensic: "ALWAYS check against latest versions of services we use web/online docs before creating instructions! ... mandatory rules / constraints, result is consistency and safety of created instructions, guides and manuals!"

Case study (Herald 2026-05-28): MTProto guide recommended VoIP + omitted recover@telegram.org pre-login email; contradicted official Telegram + gotd/td docs; could have caused permanent ban. Misguidance-by-stale-docs = §11.4 PASS-bluff at documentation layer.

(A) Pre-commit cross-reference: fetch LATEST official online docs (WebFetch/MCP/direct browsing — NEVER training data) for every operator-facing instruction doc; cite source URL+date in "## Sources verified" footer AND commit-message footer; seek secondary authoritative sources for sparse official docs. (B) Negative findings documented explicitly. (C) STALE after 6 months (90 days for risk-classified services); re-verify at vN.0.0, on breaking-change, on operator-error. (D) Risk-classified: messengers/cloud/payment/AI/code-hosting/package-managers. (E) Composes with §11.4.92 Pass 4 INDEPENDENT — cannot substitute. (F) Inheritance per §11.4.35 — restate literal 11.4.99 in every consumer's CLAUDE/AGENTS/QWEN. (G) Enforcement: missing-footer BLOCKED; stale-beyond-grace → §11.4.90 Obsolete.

Canonical authority: [Constitution.md](#) §11.4.99. Release blocker.

§11.4.100 — RETIRED. Demoted to consumer project (ATMO-Sphere video-color/visual-quality fidelity) per §11.4.17/§11.4.35 — project-specific (RK3588/MPV/Arvus), not universal. See the consuming project's Constitution/CLAUDE/AGENTS/QWEN.

§11.4.101 — Autonomous-decision-over-blocking (User mandate, 2026-05-28)

Forensic: "when working in endless working loop fully autonomously try to decide most properly about points which would block execution and wait for us. If we haven't answered now work would be blocked whole night! If possible and if that will not cause any issues make proper and most reliable and safe decision so we achieve maximal efficiency and work gets fully done!"

In autonomous / endless-loop mode (per §11.4.87) the agent **MUST** minimize operator-blocking and make the safe, reliable, reversible decision itself — work NOT stalled overnight waiting for input. §11.4.87 = keep working; §11.4.101 = HOW to clear the decision points.

Decision rule (proceed autonomously when ALL): (a) reversible OR pre-op backup per §9.2; (b) safe choice determinable from captured evidence per §11.4.6 (LIKELY ≠ determination); (c) wrong-choice blast radius bounded AND recoverable; (d) composes with §11.4 + §12 + §9. Block-only-when (BLOCK via §11.4.66 ONLY when ALL): irreversible AND high-blast-radius AND choice undeterminable from evidence — external-account state, inaccessible hardware, destructive-op-without-backup, force-push (§9.2 + §11.4.41), spending / third-party send. Operator-blocked per §11.4.21 only after this fires + self-resolution-exhaustion audit. Maximize-progress-while-blocked: a block parks one work unit, not the loop — keep progressing NON-blocked items per §11.4.87 + §11.4.94; pose-and-idle = §11.4.94 + §11.4.97 violation. Composes §11.4.6/21/40/41/66/87/94/§9.2/§12. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-101-PROPAGATION (literal 11.4.101) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.101. Release blocker. No --always-block-on-decision / --never-decide-autonomously / --skip-decision-rule / --block-without-self-resolution escape.

§11.4.102 — Systematic-debugging always-on + always-loaded skill-discovery + plugin availability (User mandate, 2026-05-29)

Forensic: "ALWAYS trigger / start the "/superpowers:systematic-debugging" skills when any issues happen! ... we **MUST** activate the skill(s) and make strongest efforts in full in depth analysis /

debugging and determine root causes ... "/using-superpowers" skill is ALWAYS loaded, applied and used! All dependencies (plugins) that Claude Code or other market places are offering MUST BE installed if these are not already available for loading and use!"

(A) On ANY spotted issue/bug/test-failure/gate-failure/regression/misalignment/inconsistency/unexpected-behaviour the agent MUST activate superpowers:systematic-debugging (or platform-equivalent) BEFORE proposing/writing/applying any fix — Iron Law: NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST. Four-phase arc: root-cause → pattern → hypothesis (prove/disprove with captured evidence) → implementation (against proven cause only). Guess-and-retry / symptom-patch / re-run-hoping-it-passes ("probably transient/flaky") without completed investigation = §11.4.102 violation; calling a failure transient/flaky/intermittent without captured forensic evidence = simultaneous §11.4.6 + §11.4.7 violation. (B) superpowers:using-superpowers (or platform-equivalent skill-discovery) ALWAYS loaded + applied at session start, consulted before any task — if ANY skill could apply (even 1%) it MUST be invoked, never improvised. (C) Every mandated skill plugin/dependency (Claude Code marketplaces or other runtime ecosystems) MUST be installed + loadable BEFORE dependent work; missing plugin blocking a mandated skill = release-blocker. Install: runtime's own path — Claude Code /plugin flow (add marketplace → install → confirm skill in available-skills list); other runtimes' documented installer. Anti-bluff: confirm via live capability list (install exit 0 ≠ skill loadable, §11.4.80 lesson). Composes §11.4.4/6/7/8/43/70/82(A)/92. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-102-PROPAGATION (literal 11.4.102) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.102. Release blocker. No --skip-systematic-debugging / --guess-and-retry-OK / --symptom-patch-permitted / --skip-skill-discovery / --plugin-optional / --missing-plugin-is-warning escape.

§11.4.103 — Continuous parallel-stream working routine (User mandate, 2026-05-29)

Forensic: "Do this working approach continuously and make it part of regular working routine and add it to the Constitution Submodule documented fully."

Promotes the proven multi-stream pattern into the project's standing default working routine (not a per-request opt-in). Load-bearing invariant: main work stream MUST always stay FREE. (A) Main stream FREE — ALL commit AND push run detached (nohup ... & + disown per §11.4.88), never blocking on push or slow mirror. (B) ≥3 parallel background streams at all times + auto-backfill

(User mandate 2026-05-29; raised from ≥ 2) — at least three sub-agent-driven streams (§11.4.70/§11.4.20, isolated §11.4.58 + §11.4.84) alongside main whenever three-plus non-contending actionable items exist; the moment any one stream is FULLY done a new stream MUST immediately start + take its place (next-highest-priority non-contending item), count NEVER drops below 3 while actionable items remain. Idle below 3 only when no remaining items OR all externally blocked (§11.4.94/97/101). Band 3-6, bounded §12.6 60% mem + §11.4.58 6-agent cap. (C) Most-critical + most-visible first; audio always top §11.4.72 + §11.4.42. (D) Safe-during-build scope only — while heavy gradle/m -j/42 GB containerised AOSP build (§12.9) runs, streams restrict to §11.4.96 SAFE catalogue; NEVER a second concurrent heavy build (§12.8). (E) Heavy anti-bluff every closure (§11.4.5/6/50/69/102; root cause proven or UNCONFIRMED:/UNKNOWN:/PENDING_FORENSICS:). (F) Idle ONLY when genuinely externally blocked (hardware/network/in-flight build-test-push) OR operator STOP OR §12 host-safety per §11.4.94(A)+§11.4.97; block parks one work unit not the loop (§11.4.101). Composes §11.4.58/70/72/87/88/89/94/96/97/101/102/42/84/§12.6-§12.9/§9.2. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-103-PROPAGATION (literal 11.4.103) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.103. Release blocker. No --block-main-stream / --synchronous-commit / --synchronous-push / --single-stream-only / --skip-parallel-streams / --serialise-actionable-work / --idle-without-queue-survey escape.

§11.4.104 — Participant identity, attribution & notification-tagging (User mandate, 2026-05-31)

Forensic: "Every supported messenger must relate messages to participants (Subscribers/Users); the same logical person may have a different username on every messenger. Workable items must carry who created them and who they are assigned to. Notifications must @-tag the right participant — but never the operator (who drives the system) and never the system agent."

MANDATORY for every consumer shipping a messenger/notification surface (Herald + flavor binaries = reference impl; others inherit §11.4.35). Detailed spec: Herald docs/design/PARTICIPANT_ATTRIBUTION.md — restated, not redefined. (A) Every messenger MUST relate messages to a **Participant** (logical Subscriber/User); SAME person MAY have a DIFFERENT username per messenger (logical subscriber: canonical messenger-neutral handle, kind $\in$ {human,agent,service}; + per-channel aliases channel/channel_user_id/@username for tagging). Canonical handle closed set: Claude (reserved system-agent sentinel; never tagged) OR a subscriber @username. (B) Operator = env var HERALD_<CHANNEL>_OPERATOR_USERNAME, not a DB flag — the one human

who drives via the agent CLI; a normal Participant whose handle equals that value. (C) Workable items MUST carry `created_by` + `assigned_to` (canonical handles): CLI-prompt→Operator; system/agent-detected→Claude; received-message→sender's resolved @username; assigned_to defaults to Operator, overridable; legacy items carry "" and MUST still parse + validate. (D) Tagging matrix: tag assigned_to if human ≠ Operator; tag created_by if human ≠ Operator ≠ Claude; NEVER tag Claude or the Operator; de-dup; resolve to channel @username, skip if absent there. (E) Anti-bluff (composes §11.4): real SQLite round-trip with the new columns byte-identical (incl. legacy fixtures WITHOUT the fields); tagging matrix proven by a truth-table + cell-flip mutation forcing FAIL; E2E real-event → real-message asserting the exact @username + a NEGATIVE case proving the Operator is NOT tagged; evidence under docs/qa/<run-id>/. Composes §11.4 + §11.4.1..16 / §11.4.5 / §11.4.69 / §11.4.50 / §11.4.91 / §11.4.93 / §11.4.95 / §1.1. Classification: universal (§11.4.17); no-messenger projects inherit latently (§11.4.96 pattern). Propagation gate CM-COVENANT-114-104-PROPAGATION (literal 11.4.104) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.104; detailed spec Herald docs/design/PARTICIPANT_ATTRIBUTION.md. Release blocker. No --skip-attribution / --no-participant-tagging / --tag-operator-anyway / --attribution-later escape.

§11.4.105 — Natural-language intent recognition & clarification (User mandate, 2026-05-31)

Forensic: "Users must NOT need to know command syntax (no COMMAND: ... prefix). They send a clear natural-language message; the System determines the intent. The System recognizes the commands it has; if none matches it infers the exact intent; if it is totally unable it replies, tags the user (@user ...), and asks to clarify precisely. We MUST always do our best to determine exact intent so we never annoy end users. This is a CORE part of the System."

MANDATORY for every consumer shipping a messenger/command surface (Herald + flavor binaries = reference impl; others inherit §11.4.35). Detailed spec: Herald docs/design/INTENT_RECOGNITION.md — restated, not redefined. (A) No required command syntax — users MUST NOT need to know any syntax (no COMMAND: prefix); they send plain natural language, the System determines the intent. (B) Three-tier resolution, first that succeeds wins: TIER 1 recognize the System's existing command set from natural language → action (confident deterministic match); TIER 2 when no command matches, infer the exact intent (LLM maps language → action), NEVER guessing; TIER 3 when neither a command nor a confident intent is determinable, REPLY to the message, TAG the sender (@username, via the §11.4.104 IdentityResolver) and ask a PRECISE clarifying question NAMING the candidate intents — no

guessing, no silent drop. (C) Never guess, never drop — a wrong action is worse than a clarifying question (composes §11.4.6); a message is NEVER silently dropped; only genuine ambiguity reaches Tier 3, which always replies-tags-and-asks. (D) Anti-bluff (composes §11.4): every tier ships unit + integration + E2E + full-automation tests with real captured evidence — Tier 1 truth-table (natural-language → action+fields + conservative negatives that MUST fall through to "no match"); Tier 3 E2E whose recorded reply body is EXACTLY @<sender> <specific question> + a NEGATIVE proving a clear command does NOT trigger clarify; a paired §1.1 mutation breaking the confidence guard (false-match) OR dropping the clarify tag MUST FAIL a test; evidence under docs/qa/<run-id>/. Composes §11.4 + §11.4.1..16 / §11.4.6 / §11.4.104 / §11.4.5 / §11.4.69 / §11.4.98 / §1.1. Classification: universal (§11.4.17); no-messenger projects inherit latently (§11.4.96 pattern). Propagation gate CM-COVENANT-114-105-PROPAGATION (literal 11.4.105) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.105; detailed spec Herald docs/design/INTENT_RECOGNITION.md. Release blocker. No --require-command-syntax / --guess-intent-ok / --skip-clarify / --drop-on-ambiguous escape.

§11.4.106 — Docs Chain — mechanical documentation/DB sync engine (Operator mandate, 2026-05-31)

Forensic: Docs Chain is the canonical mechanical enforcer of the documentation-sync mandates; consumers MUST use the engine instead of ad-hoc per-project doc-sync scripts, register chains via per-context YAML, and never accept a faked transform.

MANDATORY for every consumer. Docs Chain (the vasic-digital/docs_chain engine — a universal Go bidirectional document-and-database dependency-propagation engine) is the canonical mechanical enforcer of the documentation-sync mandates. Detailed spec: engine docs ~/Projects/docs_chain → docs/CONSTITUTION_INTEGRATION.md (distribution + inheritance + anchor mapping table) + docs/USE_CASE_CATALOGUE.md (chain recipes) — restated, not redefined. (A) Use the engine, never ad-hoc scripts — consumed by reference (the flat-layout sibling ~/Projects/docs_chain / the constitution-exposed path), inherited like §11.4.80's codegraph_* scripts: referenced, NEVER copied; ad-hoc sync_*/generate_*_summary/update_readme_doc_links scripts superseded + retired per registered context. (B) Consumer-owned contexts — the engine is project-agnostic; the consumer registers chains as data via .docs_chain/contexts/*.yaml (§11.4.28 decoupling); state.json + *.docs_chain.tmp gitignored. (C) Anchors mechanized (per CONSTITUTION_INTEGRATION mapping table): §11.4.12/.53/.45/.56/.57/.59/.60/.65/.86/.93/.95/.44 + §12.10 —

content-hash change detection (NOT mtime, §11.4.86), atomic-rename + SQLite-txn commit + rollback (§9.2), both-dirty sync → conflict-not-silent-merge (§11.4.6), verify as the deterministic CI/pre-build gate (§11.4.50), per-run captured evidence to qa-results/docs_chain/<run-id>/ (§11.4.69). (D) NOT a replacement for authoring discipline — the source author still writes the §11.4.44 revision header; the engine only keeps exports in sync. (E) Anti-bluff (composes §11.4): NEVER fakes a transform — a missing pandoc/weasyprint surfaces a typed ToolAbsentError + honest §11.4.3 SKIP-with-reason, never a fake PASS / partial write; every sync/verify carries real captured evidence. Composes §11.4 + §11.4.1..16 / §11.4.6 / §11.4.28 / §11.4.80 / §9.2 / §11.4.50 / §11.4.69 / §11.4.5 / §1.1 + the mechanized sync anchors. Classification: universal (§11.4.17); no derived-export/DB-sync projects inherit latently (§11.4.96 pattern). Status (§11.4.6): engine Phases 1–3 IMPLEMENTED+tested; CLI/YAML loader (Phase 4) + submodule distribution (Phase 6) PLANNED + OPERATOR-GATED. Propagation gate CM-COVENANT-114-106-PROPAGATION (literal 11.4.106) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: [Constitution.md](#) §11.4.106; detailed spec Docs Chain engine docs docs/CONSTITUTION_INTEGRATION.md + docs/USE_CASE_CATALOGUE.md (vasic-digital/docs_chain). Release blocker. No --ad-hoc-sync-ok / --skip-docs-chain / --fake-transform / --sync-evidence-optional escape.

§11.4.107 — Anti-bluff AV/test-validation techniques mandate (User-driven research, 2026-06-02)

Forensic (genericised, 2026-06-02): a test PASSEd on a SINGLE captured frame showing "a picture" — but it was a FROZEN/STALE frame from previously-played content (stale-producer/stuck-decoder), feature broken for the user while green; a sibling FLASHED media on the WRONG output for ~1 s before routing with no test sampling that window; a third shipped a comparator that PASSEd its own deliberately-degraded fixture (the analyzer was the bluff). §11.4.5 mandates captured evidence + a presence pass; §11.4.107 raises it to liveness + correct-routing + self-validated-analyzer.

Every test asserting audio/video output is genuinely playing/advancing MUST satisfy ALL: (1) a single captured frame is NOT proof — prove LIVE, ADVANCING frames over a window via a freeze-detection oracle (near-duplicate/freeze-detect-class OR perceptual-hash adjacent-frame distance, NOT byte-identical — byte-identity only a pre-filter); (2) an independent frame-advance counter from the platform's compositor/decoder telemetry must increase (different-domain oracle — flat ⇒ stuck decoder ⇒ FAIL even if pixels appear to move); (3) loading/buffering is a distinct

state — wait for genuine playback-start, never false-FAIL a still-loading stream nor false-PASS a spinner (timeout+unreachable ⇒ SKIP §11.4.3, timeout+reachable ⇒ FAIL); (4) not-stale-from-previous cross-check (new first frame ≠ previous last frame); (5) measured FPS / no-lost-frames within tolerance; (6) no-flash-on-the-wrong-output — high-frequency sampling of the non-target output during routing transitions, any content frame there ⇒ FAIL (content-protection regime classified, never guessed); (7) drive through the realistic feed/UI path, not deep-link shortcuts (UI-not-introspectable ⇒ operator-attended §11.4.52, never fake-PASS); (8) metamorphic relations for the oracle problem on un-owned streams (same content output-A vs output-B must match; paused ⇒ counter stops; 2× speed ⇒ ~2× advance); (9) full-reference quality metrics (SSIM/VMAF/ΔE2000) vs golden source for owned content; (10) mutation-test every analyzer with a golden-good + golden-bad fixture pair so the analyzer cannot bluff (PASS on golden-bad = bluff gate — §1.1 applied to analyzers); (11) per-channel audio RMS/loudness (EBU R128) + XRUN census (aggregate RMS misses a dead channel); (12) OCR overlay/subtitle needs a per-word confidence floor + ROI (avoids both false-FAIL and false-PASS); (13) thresholds calibrated on the project's own fixtures, not hardcoded from literature (§11.4.6). Honest gap: true photon FPS at the sink has no clean software oracle — flag it, never claim it.

4-layer §11.4.4(b): pre-build gate (CM-AV-LIVENESS-NO-FROZEN-FRAME-class + analyzer-self-validation gate wiring golden-good/golden-bad fixtures into meta-test) + runtime/on-device test + paired §1.1 mutation (single-frame-only → FAILs; analyzer PASSing its golden-bad fixture → self-validation FAILs) + HelixQA Challenge. Every PASS via `ab_pass_with_evidence` citing motion / not-stale / frame-advance / fps / loudness / metamorphic / self-validation artefacts (`video_display/audio_output/subtitle_render` §11.4.69) — never a single screenshot. Classification: universal (§11.4.17) — platform-neutral, reusable by ANY project validating media playback or any pixel/audio output; project supplies its capture mechanism + calibrated thresholds per §11.4.35. Composes §11.4.5/.6/.50/.68/.69/.85 + §11.4.2/.3/.13/.48/.52/§1.1. Propagation gate CM-COVENANT-114-107-PROPAGATION (literal 11.4.107) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.107. Release blocker. No `--single-frame-proves-playback` / `--skip-liveness` / `--byte-identical-freeze-OK` / `--no-frame-advance-counter` / `--allow-wrong-output-flash` / `--deep-link-shortcut-OK` / `--unvalidated-analyzer-OK` / `--aggregate-rms-suffices` / `--hardcoded-thresholds-OK` escape.

§11.4.108 — Four-layer fix-verification + runtime-signature-as-definition-of-done mandate (systematic-debugging Phase 4.5, 2026-06-03)

Forensic anchor (genericised, 2026-06-03): across one batch multiple fixes were "green" at every gate yet NOT working for the end user — a source-committed change passed by both the source pre-build gate AND the post-build gate never reached the boot-time command-line embedded in the deployed boot artifact (feature dead despite all-green); sibling fixes correctly built into the system image were masked by stale per-user overlay copies from a prior deployment (running code = stale shadow, not the fresh build); deployment did not wipe the mutable overlay so the shadow survived; validation ran against the stale shadow and reported green on code never exercised. Per `superpowers:systematic-debugging Phase 4.5`: each fix revealing a fresh "fixed-but-not-working" in a DIFFERENT place is ONE architectural VERIFICATION flaw, not independent bugs — patching symptoms is thrashing. A fix crosses FOUR distinct layers, and "fixed" at one does NOT imply fixed at the next: (1) SOURCE (committed — `grep-the-source` gate), (2) ARTIFACT (the change's BYTES landed in the produced artifact — image / bundle / installer / embedded command-line), (3) RUNTIME-ON-CLEAN-TARGET (active on a CLEAN/fresh deployment, the layer the end user experiences, no stale overlay shadowing it), (4) USER-VISIBLE (works for the end user — §11.4.5/§11.4.69 captured-evidence). The mandate (ALL hold): (1) **runtime-signature-as-definition-of-done** — a fix is DONE only when its declared runtime signature verifies on a CLEAN deployment; source-committed $\neq$ artifact-contains-it $\neq$ active-on-clean-target $\neq$ user-visible-working; (2) every fix declares ONE machine-checkable runtime signature (running-system property / sink-side report per §11.4.13 / §11.4.5/§11.4.69 captured-evidence / counter delta — NEVER a re-grep of the source); this registry of per-fix runtime signatures is the SINGLE SOURCE OF TRUTH for "fixed", REPLACING "a gate greps the source"; (3) gates span all four layers — source (pre-build), artifact (post-build — assert the BYTES landed, not merely that the source still contains the change), runtime-on-clean-target (post-deploy — assert the runtime signature), user-visible (§11.4.5/§11.4.69); (4) eliminate the stale-deployment / shadow layer by construction — deployment MUST yield a CLEAN state (wipe the mutable overlay) OR a pre-validation assertion MUST prove `running-artifact == built-artifact` before any validation; validation against possibly-stale state is INVALID and its PASS is a §11.4 PASS-bluff; (5) meta-rule — ≥ 3 "fixed-but-not-working" discoveries in one cycle signal an architectural VERIFICATION flaw, not three independent bugs; on the 3rd STOP patching symptoms (§11.4.4), fix the VERIFICATION pipeline, re-certify EVERY item through it on a clean target; (6) a batch is "validated" only after COMPREHENSIVE per-item runtime-signature verification on a clean baseline, NOT after the touched items' own tests pass. Classification: universal (§11.4.17)

— platform-neutral verification-pipeline disciplines reusable by ANY project that builds → deploys → validates an artifact; the project supplies its artifact format, clean-deployment / equal-artifact mechanism, and per-fix runtime-signature observables per §11.4.35. Composes §11.4.1/.2/.4/.5/.6/.27/.40/.46/.50/.52/.69/.102. Propagation gate CM-COVENANT-114-108-PROPAGATION (literal 11.4.108) + recommended per-fix gate CM-RUNTIME-SIGNATURE-REGISTRY + paired §1.1 mutations (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.108. Release blocker. No --source-green-is-done / --skip-artifact-byte-check / --validate-against-running-state / --no-clean-deployment / --skip-runtime-signature / --spot-validate-touched-only escape.

§11.4.109 — Mandatory Anti-Forgetting Enforcement: PreToolUse Guard Hook + Subagent Constitutional Preamble + Orchestrator Pre-Action Checklist (Operator mandate)

UNCONFIRMED forensic anchor (pending operator's verbatim mandate quote). Background: emulator subagents ran raw host-direct emulator/adb because the orchestrator forgot to inject the Containers-submodule rule at dispatch. A forgotten rule is not enforcement. Two-layer fix: (A) portable bash/awk PreToolUse hook (constitution/scripts/hooks/guard-forbidden-commands.sh, no jq, no project-specific paths) blocks host-direct emulator / force-push bypass / sudo / host-power at the tool-call boundary regardless of agent memory (the floor); (B) constitution/docs/AGENT_GUARDRAILS.md — canonical preamble the orchestrator pastes verbatim into every subagent dispatch + Orchestrator Pre-Action Checklist (the ceiling). Consuming projects wire the hook in .claude/settings.json or equivalent; maintain docs/AGENT_GUARDRAILS.md with SUBAGENT CONSTITUTIONAL PREAMBLE, ORCHESTRATOR PRE-ACTION CHECKLIST, and the anchor literal 11.4.109; provide a ≥20-case hermetic hook test. The hook is inherited by reference — NEVER copied locally. Gates: CM-ANTI-FORGETTING-ENFORCEMENT + CM-COVENANT-114-109-PROPAGATION + paired §1.1 mutations (gate-code = separate work item). Classification: universal (§11.4.17). Composes §11.4.6/.10/.75/.76/.78/.79/.80/.81/.84/.98/.102/§12.

Canonical authority: Constitution.md §11.4.109; ref impl constitution/scripts/hooks/guard-forbidden-commands.sh + constitution/docs/AGENT_GUARDRAILS.md. Release blocker. No --skip-pretooluse-hook / --no-guardrails-doc / --anti-forgetting-optional / --single-layer-sufficient escape.

§11.4.110 — Pre-build build-readiness verdict + change-impact clash detection mandate (operator mandate, 2026-06-03)

Forensic anchor (genericised, 2026-06-03): a fix shipped a new system-property *read* but introduced no matching security-policy grant + no property-context type entry — the read was silently denied at runtime + the feature was dead, while every pre-build gate stayed green because the gate only grepped the source file. Generalises: a change introduces a new dependency on a *second* artifact (security policy, context file, service registry, interface freeze-snapshot, symbol table, build-graph node) that the pre-build never cross-checks. This is §11.4.108's SOURCE→ARTIFACT gap shifted LEFT to pre-build — most such clashes are statically catchable from the change diff itself. The mandate (ALL hold): (1) a single deterministic **READY-FOR-BUILD verdict** gates the re-build (orchestrator refuses to start unless READY); (2) a **diff-driven change-impact + clash detector** cross-checks every newly-introduced second-artifact dependency (new property read $\Rightarrow$ property-context type + read-grant; new service $\Rightarrow$ service-context; new init service $\Rightarrow$ security label; new/changed stable interface $\Rightarrow$ freeze-snapshot updated; new native-lib dep $\Rightarrow$ module/pre-built resolves; new policy rule $\Rightarrow$ every type/attribute defined; two batch changes on the same seam $\Rightarrow$ collision acknowledged); (3) **coverage-completeness is a gate** — every changed file maps to ≥ 1 gate + ≥ 1 deployed-target test + ≥ 1 paired §1.1 mutation, baseline ratchets per §11.4.50; (4) **two-speed honesty** — grep-speed always-on gates vs REQUIRES_BUILD heavy gates (build-graph parse-only dry-run, full neverallow compilation, ABI diff) as diff-gated opt-in stages bounded per §12.6/§12.7; (5) every gate + wired analyzer is anti-bluff by paired §1.1 mutation; (6) **honest boundary** — a READY verdict proves static internal-consistency + ready-to-build, NOT that the feature works on the deployed target; the regime empties the *preventable* class, not the *all-defects* class (runtime/USER-VISIBLE remains §11.4.108's job). Classification: universal (§11.4.17) — platform-neutral pre-build-rigor disciplines reusable by ANY artifact-building project; the project supplies its property/service/policy registries, build-graph parse-only command, interface-freeze mechanism + changed-file→gate mapping per §11.4.35. Composes §11.4.1/.4/.6/.9/.27/.50/.67/.75/.92/.108. Propagation gate CM-COVENANT-114-110-PROPAGATION (literal 11.4.110) + recommended CM-READY-FOR-BUILD-VERDICT / CM-CHANGE-IMPACT-CLASH-DETECTOR / CM-COVERAGE-COMPLETENESS-GATE / CM-BUILDGRAPH-DRYRUN-WIRED / CM-SEPOLICY-NEVERALLOW-WIRED + paired §1.1 mutations (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.110. Release blocker. No --source-green-is-ready / --skip-clash-detector / --skip-coverage-gate / --no-ready-verdict / --grep-proves-neverallow / --skip-buildgraph-dryrun / --build-without-ready-verdict escape.

§11.4.111 — Resolve-by-stable-name-not-by-enumeration-index mandate (research-derived, 2026-06-03)

Forensic anchor (genericised, 2026-06-03): a platform bound an audio output to a kernel-enumerated index (card=0); a second device of a different class enumerated FIRST at boot + took slot 0, shifting the intended device to slot 1 — the static index binding now pointed at the wrong device (policy mis-attached the sink, mis-assigned its TYPE, the output switcher labelled the AV receiver "Wired Headphone" + routing collapsed to stereo). The lower layer of the SAME stack already resolved by **name** (controller-name registry) — proving the brittleness was the *index*, not the resolution capability. Generalises to every enumerated resource whose ordinal is assigned at discovery/boot/hotplug (non-deterministic across reboots, additions, topology changes). The mandate: any binding to a hardware device / resource handle / enumerated entity (audio cards, display connectors, network interfaces, storage devices, GPU render nodes, input/camera devices, container/process slots) MUST resolve by a **stable identifier** (name / UUID / serial / label / controller-name / content-hash / sink-reported identity), NOT by enumeration index / ordinal / slot, UNLESS the platform documents the ordinal as deterministically pinned AND the pin is captured + asserted in the binding. Where a stable identifier exists at one layer, every layer binding the same resource MUST use it (mixed by-name-here / by-index-there is the forbidden weak link). Honest boundary (§11.4.6): when only an ordinal exists, pin deterministically via the platform's mechanism, capture the pin in the §11.4.108 runtime signature, document residual fragility as UNCONFIRMED:-class — never silently trust an unpinned ordinal. Classification: universal (§11.4.17) — platform-neutral binding-robustness discipline; the project supplies its stable-identifier mechanism (ALSA card name, DRM connector name, NIC predictable name, block-device UUID, etc.) + the layers that must agree per §11.4.35. Composes §11.4.6/.8/.69/.108/.110. Propagation gate CM-COVENANT-114-111-PROPAGATION (literal 11.4.111) + recommended CM-RESOLVE-BY-NAME-NOT-INDEX + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.111. Release blocker. No --allow-index-binding / --ordinal-is-stable-enough / --skip-name-resolution / --trust-unpinned-index escape.

§11.4.112 — Structural-impossibility won't-fix classification mandate (research-derived, 2026-06-03)

Forensic anchor (genericised, 2026-06-03): deep research (§11.4.8) into relocating protected (content-protection / secure-surface) video to a secondary display PROVED — from authoritative platform/HDCP docs + reproducible captured behaviour (a secure surface is blanked on any output lacking the secure flag; mirror/screenshot of a secure layer returns black) — that the goal is **structurally impossible by platform design**, not a missing feature. Without a durable classification it is re-investigated every cycle (re-read the same sources, re-run the same probes, re-derive the same impossibility — compounding wasted effort). The mandate: when deep research per §11.4.8 PROVES (cited authoritative sources AND, where applicable, reproducible captured evidence) a goal is structurally impossible on the target platform (forbidden by platform design / hardware-protocol constraint / documented kernel-or-API limitation — NOT merely unimplemented or hard), the goal MUST be: (1) classified Won't-fix + closed per §11.4.90 with reason structurally-impossible; (2) documented with the impossibility evidence — cited source URLs (per §11.4.99) + reproducible probe/captured evidence — in the tracker entry + relevant docs/ guide; (3) NOT re-attempted in future cycles — a reopen MUST cite NEW evidence the constraint changed (per §11.4.34 + §11.4.7), never re-derive the same impossibility; (4) paired with the correct posture — the entry states what the project DOES instead, so "impossible" is never confused with "broken/unhandled". Honest boundary (§11.4.6): reserved for *proven* impossibility — "could not find a way" / "very hard" / "no time" are Operator-blocked (§11.4.21) or open work, NOT won't-fix; mislabelling them is a §11.4 planning-layer bluff. A future platform change can make the impossible possible (durable, not eternal). Classification: universal (§11.4.17) — platform-neutral effort-conservation + honesty discipline; the project supplies the impossible goal, its constraint + cited evidence per §11.4.35. Composes §11.4.6/.7/.8/.34/.90/.99. Propagation gate CM-COVENANT-114-112-PROPAGATION (literal 11.4.112) + recommended CM-WONT-FIX-STRUCTURAL-IMPOSSIBILITY + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: [Constitution.md](#) §11.4.112. Release blocker. No --wont-fix-without-proof / --reattempt-closed-impossible / --skip-impossibility-evidence / --impossible-equals-broken escape.

§11.4.113 — Absolute no-force-push + merge-onto-latest-main mandate (User mandate, 2026-06-03)

Forensic anchor — verbatim user mandate (2026-06-03): "Any force-push is strictly forbidden! We must for every Submodule take as a base latest commit on Submodule's main (or master)

branch, then on top of it carefully to merge all changes that have to be pushed! Once all merging is carefully done we perform commit and push to all Submodule's upstreams!" Force-push is STRICTLY FORBIDDEN with NO exception — `git push --force, --force-with-lease, +<ref>`, any history-rewriting overwrite of a remote ref — against EVERY repository this Constitution governs (main repo, this constitution submodule, every owned + nested submodule, every upstream). No operator-approval path, no "after merge-first audit" path. Mandated 6-step integration for any repo/submodule with commits to publish OR diverged mirrors: (1) `git fetch --all --prune --tags` all remotes; (2) base = LATEST commit on canonical main/master (most-advanced mirror tip); (3) carefully MERGE every change on top — union, preserve BOTH sides, NEVER `-s ours` / rebase / reset that drops commits (§9 no-commit-loss); (4) resolve conflicts carefully — no markers, no file dropped, gates/tests still pass; (5) commit the merge (stage only intended files, NEVER `git add -A` in a submodule per §11.4.30); (6) push to ALL upstreams — each is a fast-forward because the merge commit descends from every mirror tip, so NO force is needed (an upstream still rejecting → return to step 1 for it, merge its new tip, re-validate, re-push). TIGHTENS §11.4.41 / §11.4.71 / §9.2 / CONST-043 — force-push escape hatch REMOVED: even WITH operator approval, even after a clean merge-first audit, force-push is forbidden, because the merge-onto-latest-main path is always available so force is never necessary; those clauses' merge-first/fetch-first machinery stays, only their terminal "...then force-push" step is struck. Classification: universal (§11.4.17). Composes §2.1 / §9 / §9.2 / §11.4.4 / §11.4.6 / §11.4.26 / §11.4.37 / §11.4.40 / §11.4.41 / §11.4.71 / §11.4.88 / CONST-043. Propagation gate CM-COVENANT-114-113-PROPAGATION (literal 11.4.113) + recommended CM-NO-FORCE-PUSH-ABSOLUTE (scan tracked scripts/hooks for `push --force / --force-with-lease / push +<ref> + reject`; §11.4.109-class PreToolUse guard blocks the class) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.113. Release blocker. No `--force / --force-with-lease / --allow-force-push / --force-push-authorised / --skip-merge-onto-main` escape.

§11.4.114 — Last-known-good-tag regression isolation mandate (1.1.8-dev remediation, 2026-06-03)

When a previously-working feature/behaviour is observed broken, the FIRST diagnostic action MUST be to identify the last release tag (or commit/build/deploy) at which it was KNOWN-GOOD and diff/bisect the broken state against it — BEFORE any open-ended root-cause hunt or speculative fix. The known-good revision is the regression oracle: bounds the search to commits between good and now (`git diff <good-tag>..HEAD --stat` of the feature's files = captured evidence), marks it a regression REPAIR not from-

scratch design, gives each suspect file a behavioural oracle. An operator-volunteered known-good tag is load-bearing → act on it first. Default to SURGICAL forward-fix (keep post-good-tag features, revert ONLY the broken sub-part) over wholesale revert (which loses the batch's other working features) unless operator prefers wholesale. Honest boundary (§11.4.6): "it worked before" is a HYPOTHESIS until the tag is identified AND the feature confirmed working there; "probably regressed last batch" without the diff is a guess; a never-worked feature is not a regression. Classification: universal (§11.4.17). Composes §11.4.4 / §11.4.6 / §11.4.7 / §11.4.40 / §11.4.43 / §11.4.102 / §11.4.108. Propagation gate CM-COVENANT-114-114-PROPAGATION (literal 11.4.114) + recommended CM-REGRESSION-ISOLATED-AGAINST-KNOWN-GOOD + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: [Constitution.md](#) §11.4.114. Release blocker. No --skip-known-good-diff / --root-cause-from-scratch / --assume-regression-source / --wholesale-revert-without-isolation escape.

§11.4.115 — RED-baseline-on-the-broken-artifact + polarity-switch mandate (1.1.8-dev remediation, 2026-06-03)

Strict refinement of §11.4.43's RED step. Every RED test MUST REPRODUCE the defect on the CURRENT pre-fix artifact (the actual broken build/deploy), capturing positive evidence per §11.4.5 / §11.4.69 / §11.4.107 that the defect is genuinely present — never a synthetic failure the fix is then written to agree with. The SAME source MUST carry a single polarity switch (env flag / parameter, canonical RED_MODE, default 1 = reproduce-and-assert-defect-present) that flipped to 0 post-fix becomes the GREEN regression-guard asserting the defect is ABSENT. One source, two roles: the bug-catcher IS the regression-guard; no separate happy-path test is the primary guard (that only proves the test agrees with the fix — the §11.4.43 PASS-bluff). RED-on-broken then GREEN-on-fixed (clean target per §11.4.108) both captured. Honest boundary (§11.4.6): a RED run that does NOT fail on the broken artifact is a finding (close per §11.4.7 with negative evidence, or fix the blind test). Classification: universal (§11.4.17). Composes §11.4.1 / §11.4.2 / §11.4.5 / §11.4.69 / §11.4.107 / §11.4.4 / §11.4.7 / §11.4.43 / §11.4.50 / §11.4.108 / §11.4.114. Propagation gate CM-COVENANT-114-115-PROPAGATION (literal 11.4.115) + recommended CM-RED-POLARITY-SWITCH-PRESENT + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: [Constitution.md](#) §11.4.115. Release blocker. No --green-only-test / --skip-red-reproduction / --separate-happy-path-suffices / --synthetic-red-OK escape.

§11.4.116 — Real-time conductor↔autonomous-test-framework sync channel mandate (1.1.8-dev remediation, 2026-06-03)

Any autonomous, long-running test/QA/validation framework an external orchestrator (conductor agent / operator) depends on for real-time decisions MUST expose a real-time sync channel: (1) a structured append-only event stream (JSONL or equivalent, one event per line, never rewritten — session-start / phase-transition / per-test-or-challenge-start / captured-evidence-path / external-call (LLM/vision/sink-probe) / error / per-item-verdict); (2) an atomically-rewritten status snapshot (single small file, write-temp-then-rename so no torn read) with current session/phase/item + counters + last verdict. Verdicts use the closed PASS / FAIL / SKIP / OPERATOR-BLOCKED vocabulary (§11.4.45). The conductor tails it live → real-time sync, §11.4.4 interrupt on a fresh defect, never idles blindly (§11.4.94 / §11.4.97). Anti-bluff: every verdict event MUST carry the evidence path that backs it (§11.4.69) — a PASS event with no evidence path is a channel-layer bluff; a snapshot PASS contradicted by a stream with no evidence event → treat as FAIL; no start-event → no verdict-event. Owned project-agnostic submodule (§11.4.28): channel stays project-neutral, the consumer registers its data via the public API at runtime, never hard-coded. Classification: universal (§11.4.17). Composes §11.4.4 / §11.4.5 / §11.4.69 / §11.4.27 / §11.4.28 / §11.4.45 / §11.4.52 / §11.4.89 / §11.4.94 / §11.4.97. Propagation gate CM-COVENANT-114-116-PROPAGATION (literal 11.4.116) + recommended CM-AUTONOMOUS-FRAMEWORK-SYNC-CHANNEL + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.116. Release blocker. No --no-sync-channel / --end-of-session-report-suffices / --verdict-without-evidence-path / --torn-status-write-OK escape.

§11.4.117 — Computer-vision / OCR pixel-oracle fallback for non-introspectable UIs mandate (1.1.8-dev remediation, 2026-06-03)

Any test that drives a UI control OR asserts on-screen content MUST NOT assume the accessibility/semantic/DOM hierarchy is the source of truth for what the user sees. When the hierarchy is blank/partial/known-unreliable (TV-Compose, leanback, canvas/SurfaceView/GL, games, custom-rendered UIs), the test MUST fall back to a PIXEL ORACLE: (1) DRIVE input by computer-vision template-match (locate a control by rendered appearance → tap coordinates), not a node id; (2) ASSERT content by ROI OCR (read the rendered text the user reads) with a per-word confidence floor + ROI per §11.4.107(12), not a hierarchy text attribute that may be absent/stale. The tool MUST both drive input AND read pixels — a hierarchy-only tool is NOT a content oracle. Makes §11.4.52's

"near-empty hierarchy → INFEASIBLE" constructive: pixel-drive, not only SKIP. Anti-bluff (§11.4.107(10)): the CV/OCR analyzer is self-validated (golden-good PASS, golden-bad FAIL, wired into meta-test); thresholds calibrated on the project's own frames, not hardcoded (§11.4.6). Honest boundary: BOTH hierarchy blank AND pixel oracle infeasible (secure surface black-captures §11.4.112, geo-unreachable) → SKIP-with-reason §11.4.3 + tracked operator-attended migration §11.4.52, never fake PASS. Classification: universal (§11.4.17). Composes §11.4.5 / §11.4.6 / §11.4.48 / §11.4.49 / §11.4.52 / §11.4.107 / §11.4.112. Propagation gate CM-COVENANT-114-117-PROPAGATION (literal 11.4.117) + recommended CM-CV-OCR-PIXEL-ORACLE-FALLBACK + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.117. Release blocker. No --hierarchy-is-content-oracle / --skip-pixel-fallback / --unvalidated-ocr-OK / --hardcoded-ocr-threshold-OK escape.

§11.4.118 — Discovery-pressure to confirm known-issue-set completeness mandate (1.1.8-dev remediation, 2026-06-03)

A remediation/release cycle MUST NOT treat "every reported defect is fixed" as "the build is good" — the reported set is biased by what the operator happened to test ("we only see what we test"). After/alongside fixing the reported set, run a discovery + stress pass across ALL target devices/environments that deliberately exercises subsystems/journeys/edge-cases BEYOND the reported defects — to CONFIRM the reported set is the COMPLETE critical set + surface unreported defects before the user does. The pass MUST produce PROVABLE coverage: an enumerated list of subsystems/journeys/stress-scenarios exercised, each with outcome (no-new-issue, or a new tracker entry per §11.4.15 / §11.4.16). "We found no other issues" is a §11.4 bluff unless paired with "here is the enumerated set we exercised" — absence of evidence of looking is not evidence of absence. New findings trigger §11.4.4 interrupt + §11.4.114/§11.4.115 isolation→RED→fix. Honest boundary (§11.4.6): discovery reduces the unknown-unknown surface, does not prove zero remaining defects — earned claim is "reported set + enumerated discovery set addressed," un-exercised subsystems stated as honest coverage gaps (§11.4.3), never silently implied clean. Classification: universal (§11.4.17). Composes §11.4.4 / §11.4.5 / §11.4.69 / §11.4.25 / §11.4.40 / §11.4.42 / §11.4.52 / §11.4.85 / §11.4.114 / §11.4.115 / §11.4.119. Propagation gate CM-COVENANT-114-118-PROPAGATION (literal 11.4.118) + recommended CM-DISCOVERY-COVERAGE-ENUMERATED + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.118. Release blocker.
No --reported-set-suffices / --skip-discovery-pass / --no-issues-
without-coverage-list / --assume-complete escape.

§11.4.119 — Single-resource-owner partitioning for parallel hardware testing mandate (1.1.8-dev remediation, 2026-06-03)

Strict refinement of §11.4.58 / §11.4.103 for hardware contention. When multiple parallel streams exercise SHARED hardware / any exclusive-access resource (a media-playback path, a single HDMI/ audio sink, an exclusive device handle, a serial/JTAG line, a GPU under exclusive capture), exactly ONE stream MUST own each such resource at a time. The owner drives it (playback, input injection, capture); every other concurrent stream targeting it MUST be READ-ONLY (passive probes — dumphys / /proc / /sys reads / sink-side network probes / log tails). Parallelism partitioned by resource: distinct devices/sinks/handles fully concurrent (stream-per-device), but the same device's exclusive resource is single-owner. Ownership enforced by an advisory lock/token (§11.4.58 L3 + the hardware analogue of §11.4.84 quiescence), event-driven (claim when the resource frees, release the moment work completes so the next queued stream claims it per §11.4.103 auto-backfill). Why: concurrent drivers of one exclusive resource produce CROSS-CONTAMINATED evidence (sink reports the wrong stream's audio, foreground belongs to whichever am start landed last, input events interleave) — a PASS under contention is a §11.4 evidence-integrity bluff. Honest boundary (§11.4.6): a "read-only" probe stream MUST issue NO state-changing command against a device it does not own; a non-partitionable resource → streams serialize (single-owner over time), not run concurrently and hope. Classification: universal (§11.4.17). Composes §11.4.5 / §11.4.69 / §11.4.13 / §11.4.50 / §11.4.58 / §11.4.82 / §11.4.84 / §11.4.103 / §11.4.118. Propagation gate CM-COVENANT-114-119-PROPAGATION (literal 11.4.119) + recommended CM-SINGLE-RESOURCE-OWNER-PARTITION + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.119. Release blocker.
No --allow-concurrent-resource-drivers / --skip-ownership-lock / --
read-only-may-mutate / --contended-evidence-OK escape.

§11.4.120 — Fix-breaks-its-own-gate reconciliation mandate (1.1.8-dev remediation, 2026-06-03)

When a correct fix causes a pre-existing gate/test to FAIL because that gate asserted the OLD (now-removed-or-changed) behaviour, the gate FAIL is the CORRECT signal the fix landed — it MUST NOT be suppressed by either forbidden response: (1) FAKE-PASSING the gate (edit to always pass, weaken to a tautology, or

delete — a §11.4 gate-layer bluff + a §11.4.84 mutation-residue risk); (2) REVERTING the correct fix to satisfy the stale gate (re-introducing the defect). Required response = RECONCILIATION: rewrite the gate to assert the NEW mechanism the fix introduced, backed by captured evidence of the new correct behaviour, AND update its paired §1.1 mutation so the mutation breaks the NEW invariant. Discriminator vs bluffing: after reconciliation gate + mutation still form a valid §1.1 pair (mutate the new invariant → gate FAILs); a bluffed gate's mutation no longer makes it FAIL. The reconciliation MUST be a visible evidence-cited change, never a silent assertion-weakening. Honest boundary (§11.4.6): a post-fix gate FAIL is NOT automatically "stale gate" — investigate per §11.4.102 first; the FAIL may be the gate correctly catching a REGRESSION the fix introduced (then the FIX is wrong). Reconcile ONLY when investigation PROVES the gate asserted old-correct-now-removed behaviour AND the new behaviour is intended + evidence-confirmed. Classification: universal (§11.4.17). Composes §11.4.1 / §11.4.4 / §11.4.6 / §11.4.84 / §11.4.102 / §11.4.108 / §1.1. Propagation gate CM-COVENANT-114-120-PROPAGATION (literal 11.4.120) + recommended CM-GATE-RECONCILED-NOT-FAKE-PASSED + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.120. Release blocker. No --fake-pass-stale-gate / --revert-fix-for-gate / --weaken-assertion-to-pass / --delete-failing-gate escape.

§11.4.121 — No-commit-while-build-writes-tracked-artifacts mandate (1.1.8-dev remediation, 2026-06-03)

A commit (especially `git add -A` / any broad stage) MUST NOT run while a build/packaging/generation step is actively writing artifacts INTO tracked (version-controlled) directories — it races the writer and stages a PARTIAL or stale artifact (a §11.4.108 SOURCE→ARTIFACT integrity failure landed in version control). The commit MUST be deferred until the build step that writes tracked artifacts has COMPLETED, so the tree is quiescent at the artifact layer AND the committed artifacts are the FRESH, whole outputs (no stale pre-rebuild artifact, no half-written file). Before committing tracked build outputs, verify the writing step finished (process exit / completion marker / per-artifact mtime ≥ build-start). Build-output analogue of §11.4.84 (no commit while a mutation gate is in flight): both close "commit captures transient non-final tree state" at two write-sources. Gitignored build outputs (out/ / dist/) are exempt — the race doesn't apply. Honest boundary (§11.4.6): "the build probably finished" is not "the build finished" — verify with a completion signal; source changes NOT touching the build's tracked-artifact dirs are fine mid-build. PROJECT-SPECIFIC instance (which tracked dir + which build steps write it) recorded in the consuming project's own governance per §11.4.35. Classification: universal (§11.4.17). Composes §11.4.6 /

§11.4.30 / §11.4.58 / §11.4.84 / §11.4.88 / §11.4.96 / §11.4.103 / §11.4.108. Propagation gate CM-COVENANT-114-121-PROPAGATION (literal 11.4.121) + recommended CM-NO-COMMIT-DURING-ARTIFACT-WRITE + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.121. Release blocker. No --commit-during-build / --stage-partial-artifact-OK / --assume-build-finished / --skip-build-completion-check escape.

§11.4.122 — No-silent-removal-of-existing-components-without-operator-confirmation mandate (User mandate, 2026-06-03)

Forensic anchor — verbatim user mandate (2026-06-03):

"Never ever remove any application, system component or service from already existing codebase / System without interactively asked question to us! THIS IS MANDATORY RULE / CONSTRAINT!" Case study (FACT): in the 1.1.8-dev burn-down, F2 (an Apple-TV-class app) + F4 (a Huawei HMS component) were removed WITHOUT asking; the operator reversed both.

No application / system component / service / package / feature / driver / module / library / prebuilt asset — any already-existing end-user capability — may be removed (deleted, dropped from the package set, disabled-into-non-shipping, un-bundled, de-listed) from the existing codebase / shipped System WITHOUT FIRST interactively asking the operator (via the §11.4.66 interactive mechanism — AskUserQuestion on Claude Code, never free-text, never silent, never an autonomous decision) and receiving an EXPLICIT keep-or-remove decision. A removal is: a deletion from the package/service/module manifest set whose NET EFFECT is "a capability that shipped before no longer ships." Adding / replacing-with-operator-approved-equivalent / fixing is NOT a removal; when uncertain, treat AS a removal and ask (§11.4.6 no-guessing). A silent removal is a release blocker regardless of rationale (dedup / "broken anyway" / geo-restricted / incompatible / superseded). The tracked DROP path: ask → operator approves → mark item Obsolete (→ Fixed.md) with reason feature-removed + operator approval citation (§11.4.90) → then remove. Classification: universal (§11.4.17). Composes §11.4.66 / §11.4.101 (removal of a live capability is exactly the high-blast-radius class §11.4.101 reserves for an operator question, never an autonomous decision) / §11.4.90 / §11.4.112 / §11.4.6 / §11.4.40 / §11.4.42. Propagation gate CM-COVENANT-114-122-PROPAGATION (literal 11.4.122) + recommended gate CM-NO-SILENT-COMPONENT-REMOVAL + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.122. Release blocker. No --remove-without-asking / --silent-removal / --autonomous-removal-OK / --dedup-removal-exempt / --it-was-broken-anyway escape.

§11.4.123 — Rock-solid-proof-or-deep-research mandate (User mandate, 2026-06-03)

Forensic anchor — verbatim user mandate (2026-06-03):

"Every single reported issue MUST BE fully and 100% validated with rock solid proofs! Nothing can be considered fixed or completed without hard evidence! No false results or bluff(s) of any kind is allowed! If we are not sure on how to achieve full testing, validation and verification of something we MUST ALWAYS perform deep web research for all possible data (articles, documentation, guides, and other resources) and opensourced codebases which we can use to solve our problems and perform testing with validation and verification which produces rock-solid evidence(s) and leaves no space for false results or any kind of bluff!" Case study (FACT): in the 1.1.8-dev remediation the validation method for FLAG_SECURE secondary-display capture (black frames) + non-introspectable streaming-app UIs (blank accessibility tree) was unclear; deep web research (docs/research/testing_frameworks_20260603/) yielded the CV/OCR/liveness/sink-probe oracle stack (now §11.4.107/§11.4.112/§11.4.117), making rock-solid evidence possible — "unclear how to validate" is a research trigger, never a bluff licence.

Every reported issue / fix / claimed completion MUST be 100% validated with rock-solid CAPTURED proof (§11.4.5/§11.4.69/§11.4.107) before it is marked fixed/implemented/completed (§11.4.33). Nothing is fixed/complete without hard captured evidence — metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden (§11.4/§11.4.1); no false results, no bluff of any kind. Operative addition: when UNSURE how to fully test/validate/verify something (no obvious evidence-producing method, OR the candidate yields only metadata/config/absence-of-error evidence), the agent MUST ALWAYS FIRST perform deep web research per §11.4.8+§11.4.99 (official docs, articles, guides, vendor refs, standards, issue trackers, reusable open-source codebases) to DISCOVER or BUILD an evidence-producing validation method that leaves no room for a false result. Declaring "untestable"/"not automatable" or accepting a metadata-only PASS WITHOUT first exhausting this deep-research path is itself a §11.4.123 violation (same severity as a PASS-bluff); the cited research (URLs + the method, OR "NO external solution found — original work" per §11.4.8) is the proof the path was exhausted. Only after that research genuinely fails may the item be PENDING_FORENSICS:/Operator-blocked (§11.4.21)/structurally-impossible won't-fix (§11.4.112) with the cited research as earned evidence. Classification: universal (§11.4.17). Composes §11.4.5/§11.4.6/§11.4.8/§11.4.52/§11.4.69/§11.4.99/§11.4.107/§11.4.118/§11.4.21/§11.4.112. Propagation gate CM-COVENANT-114-123-PROPAGATION (literal 11.4.123) + recommended gate CM-ROCK-SOLID-PROOF-OR-RESEARCH + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.123. Release blocker.
No --metadata-pass-suffices / --skip-proof / --untestable-without-research / --config-only-closure-OK / --bluff-when-unsure escape.

§11.4.124 — Dead/unwired-code investigate-before-remove mandate (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04):

"Before removing any seemingly-dead (zero-importer / unwired) codebase, we MUST investigate via git history where/how it was originally used and how it became dead. Removal is permitted ONLY when we have captured PROOF it is genuinely no longer needed — and that removal MUST be its own separate commit with a proper descriptive message. If there is no such proof, the code MUST be investigated for where/how it should be wired in properly, and any missing or unwired tests MUST be added. We MUST ALWAYS be extra careful with any codebase removal."

"Zero importers / never called / unwired ⇒ dead ⇒ delete" is a GUESS (§11.4.6), never a finding — a "no references" result proves only current non-reference, not genuinely-unneeded. Before removing ANY seemingly-dead element (zero-importer / never-called / unwired function / method / type / file / module / package / asset / config / build target) the agent MUST FIRST investigate via git history (git log --follow, git log -S/-G pickaxe, blame on the deleted call-site) and capture as FACT: (1) WHERE/HOW it was originally wired in, (2) WHEN/HOW it became dead (call-site deleted deliberately / by mistake-regression / never-completed / refactored-unreachable), (3) whether "no references" is real OR a hidden reference the static tool cannot see (reflection / dynamic dispatch / build-tags / codegen / DI / plugin registry / FFI / config-driven). Removal is permitted ONLY with captured PROOF the element is genuinely no longer needed; that removal MUST be its OWN SEPARATE COMMIT (independently reviewable + revertible per §11.4.84/§11.4.92) with a descriptive message citing the git-history evidence (+ §11.4.122 operator-confirmation if it is an end-user capability; tracked via §11.4.90 obsolete (→ Fixed.md)). With NO such proof the element MUST NOT be removed — instead investigate WHERE/HOW to wire it in properly (restore a mistakenly-deleted call-site per §11.4.114; finish never-completed wiring) AND add any missing / unwired tests (§11.4.27/§11.4.43/§11.4.115). ALWAYS be extra careful with removal: when uncertain, default to NOT removing (investigate + wire + test) per §11.4.6 + §11.4.101 + §11.4.122; "probably dead" is never sufficient — the bar is captured proof. Classification: universal (§11.4.17). Composes §11.4.6 / §11.4.8 / §11.4.84 / §11.4.90 / §11.4.92 / §11.4.101 / §11.4.114 / §11.4.122 / §11.4.27 / §11.4.43 / §11.4.115. Propagation gate CM-COVENANT-114-124-PROPAGATION (literal 11.4.124) + recommended gate CM-DEAD-CODE-INVESTIGATE-BEFORE-

REMOVE (a net-deletion commit must be removal-only + cite the git-history investigation OR be part of a tracked Obsolete item) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.124. Release blocker. No --zero-importers-means-dead / --delete-unwired-on-sight / --skip-git-history-investigation / --remove-without-proof / --bundle-removal-with-other-work escape.

§11.4.125 — Code-review-agent gate before pre-build + main build (mandatory multi-layer review) (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04):

"After all fixes/changes/implementations are done, BEFORE running pre-build tests and the main build, dispatch code-review agent(s) that analyze all work done + all existing data/facts + the existing codebase + current git history to determine quality, safety, and whether the fixes/changes will REALLY work; they MUST validate and verify that every test covering the fixes/changes genuinely validates the work with NO chance of false results or bluff of any kind. Any finding MUST be fixed, polished, improved, and covered with additional tests before the build proceeds. Multiple strong layers of checks."

After all fixes / changes / implementations in a batch are done, and BEFORE running the pre-build test sweep AND the main (artifact) build (for ANY project), the agent MUST dispatch one or more dedicated code-review agent(s) (subagent-driven by default per §11.4.70/§11.4.20) performing a multi-layer review that: (1) analyzes ALL work done in the batch (every fix/change + its source diff + stated intent); (2) analyzes ALL existing data + facts (captured evidence per §11.4.5/§11.4.69/§11.4.107, tracker entries, prior findings, the §11.4.108 runtime-signature registry); (3) analyzes the existing codebase (blast radius per §11.4.92, cross-feature interaction, contract integrity of every dependency); (4) analyzes current git history (what each change touched, how it composes with concurrent/recent work, whether it reproduces a known-broken pattern per §11.4.114/§11.4.124); (5) determines quality + safety + will-it-REALLY-work (robust + not error-prone — no solve-A-create-B; no host/data/security regression; genuinely delivers the end-user-visible behaviour per §11.4/§107); (6) validates + verifies the tests covering the work — every covering test genuinely exercises the work-under-test and catches its negation, with ZERO chance of a false result or bluff (a test that PASSES on broken-for-the-user work, a metadata-only/config-only/absence-of-error/grep-without-runtime assertion, or a gate whose paired §1.1 mutation does not make it FAIL is a finding). Any finding (defect / error-prone change / safety risk / will-not-really-work / bluff-or-false-result-capable test / missing-coverage gap) MUST be fixed, polished, improved, and covered with additional tests (four-layer

per §11.4.4(b), TDD-RED-first per §11.4.43/§11.4.115) BEFORE the pre-build sweep + main build proceed; the review iterates (re-review after each remediation) until no blocking findings remain. The review is itself anti-bluff (its conclusions are captured evidence per §11.4.5/§11.4.69; a rubber-stamp review of a defective batch = PASS-bluff). It is one of MULTIPLE STRONG LAYERS — complementing, never replacing, the §1 pre-build sweep, §11.4.92 multi-pass (author-side self-review; §11.4.125 adds the structurally-separated reviewer seam per §11.4.70), §11.4.108 four-layer fix-verification, §11.4.110 build-readiness verdict, and the post-build / runtime-on-clean-target / user-visible layers. Composes §11.4 / §11.4.1 / §11.4.4 / §11.4.6 / §11.4.40 / §11.4.43 / §11.4.50 / §11.4.70 / §11.4.20 / §11.4.92 / §11.4.102 / §11.4.107 / §11.4.108 / §11.4.110. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-125-PROPAGATION (literal 11.4.125) + recommended gate CM-CODE-REVIEW-GATE-BEFORE-BUILD (build starts only with a fresh code-review-completed marker for the current batch, produced after the last fix + before the pre-build sweep + main build) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.125. Release blocker. No --skip-code-review / --build-without-review / --no-review-gate / --review-optional / --trust-the-author escape.

§11.4.126 — Default autonomous-loop working mode from first prompt (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04):

"Make sure that you continue work in endless fully autonomous loop, do not stop until new fully validated and verified version (tag) is created and published (all submodules and main repo) or IN A CASE OF some other main stream work until it is fully completed with all side work streams and nothing else is left in our working queue! THIS MUST BE ALWAYS the default working mode without us asking you! We tend to achieve ABSOLUTE EFFICIENCY, with this and all other projects which will incorporate this MANDATORY RULE / CONSTRAINT!!! This way of (your) working will be ALWAYS applied / followed / executed / fully respected, as soon as we assign / send first request (prompt) in the session! This stops only if we explicitly say so or nothing is left to be done in current working scope (release that will come / upcoming version)!!! Any mimicking (imitation) of this behavior / rules / mandatory constraints, false results or any kind of bluff(s) is ABSOLUTELY FORBIDDEN!!!"

The endless fully-autonomous loop is the **DEFAULT working mode**, engaged automatically the moment the operator sends the FIRST request / prompt of a session — the operator MUST NOT have to ask for it, request it, restate it, or re-enable it per session. §11.4.87 framed the endless-loop covenant as an explicit-

instruction opt-in; §11.4.126 is the **capstone** that promotes the same covenant to always-on: from the first prompt onward, every agent operates in the §11.4.87 loop discipline as the standing default, with §11.4.94 zero-idle, §11.4.97 maximum-idle-use, §11.4.101 autonomous-decision-over-blocking, and §11.4.103 continuous-parallel-stream all engaged by default — no per-session activation handshake. The continuation contract: the loop continues until ONE of two terminal conditions holds — (A) **Release scope** — a new, fully-validated-and-verified version (tag) is created AND published across all owned submodules AND the main repo to all configured remotes (per §2.1 + §11.4.40 + §11.4.113); OR (B) **Non-release main-stream scope** — the main-stream goal is fully completed AND every side work stream is done AND the working queue holds nothing left for the current scope. Until (A) or (B) holds, the agent MUST keep working per §11.4.42 / §11.4.72 / §11.4.94 / §11.4.103. The loop STOPS ONLY on: (1) the operator explicitly saying so (STOP / pause / end); (2) nothing left to do in the current working scope with the queue genuinely empty per the (A)/(B) terminal conditions; (3) a §12 host-session-safety demand. Idle-while-blocked parks one work unit, it does not stop the loop (§11.4.101 + §11.4.94 + §11.4.97). Goal — ABSOLUTE EFFICIENCY; applies to this project AND every project that incorporates this Constitution. Anti-bluff: mimicking / imitating this loop behaviour, narrating continuation without performing it, fabricating progress, or emitting false / bluff results of ANY kind is ABSOLUTELY FORBIDDEN (composes §11.4 / §11.4.1 / §11.4.2 / §11.4.5 / §11.4.6 / §11.4.50 / §11.4.69 / §11.4.107); a report claiming the loop ran while no real work / no captured evidence was produced is a §11.4 PASS-bluff at the operating-mode layer. Classification: universal (§11.4.17). Composes with §11.4.87 / §11.4.94 / §11.4.97 / §11.4.101 / §11.4.103 / §11.4.66 / §11.4.6 / §11.4.40 / §11.4.42 / §11.4.72 / §11.4.113 / §2.1 / §12. Propagation gate CM-COVENANT-114-126-PROPAGATION (literal 11.4.126) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: Constitution.md §11.4.126. Release blocker. No --ask-before-continuing / --single-turn-only / --not-default-loop / --mimic-OK escape.

§11.4.127 — Session-handoff resumption-prompt mandate (User mandate, 2026-06-06)

Forensic anchor — verbatim user mandate (2026-06-06):

"make sure that in situations like this now when new session is needed you ALWAYS prepera such sentence - which will be valid for particular moment and the phase of the project and enough for work to continue."

When a fresh session is needed (context limits / degradation) OR the operator asks whether a new session is needed / requests a handoff, the agent MUST ALWAYS prepare + proactively provide a

ready-to-paste **resumption prompt valid for that EXACT moment + project phase** — self-contained for ZERO-loss resume. Two variants: SHORT first-sentence ("Read <handoff docs>, then continue <goal> ...") + FULL block on demand. MUST (1) point to live handoff docs (.remember/remember.md if present + docs/CONTINUATION.md §12.10, read FIRST + git fetch --all); (2) state PHASE + NEXT action + terminal goal; (3) embed exact live-state anchors (build IDs/MD5, device/target serials, commit HEAD, in-flight PIDs+log paths, captured-evidence paths); (4) restate binding constraints (anti-bluff §11.4, no-force-push §11.4.113, exact version/naming, hardware/target gotchas); (5) be MOMENT-VALID never generic. Handoff docs current BEFORE the prompt (§12.10). Missing/stale/generic = §11.4.127 violation. Composes §12.10 / §11.4.6 / §11.4.66 / §11.4.87 / §11.4.103 / §11.4.126. Classification: universal (§11.4.17). Gate CM-COVENANT-114-127-PROPAGATION (literal 11.4.127) + paired §1.1 mutation.

Canonical authority: Constitution.md §11.4.127. Release blocker. No --skip-handoff-prompt / --generic-prompt-OK / --no-resumption-sentence / --handoff-without-state escape.

§11.4.128 — Always-on device-recording mandate (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): ALWAYS live-record all available data from all test/manual-testing devices, EXTRA carefully (never harm device / performance / no side effects); raw NOT processed without need (token-conscious); raw ALWAYS git-ignored + code-intelligence-excluded; only curated evidence committed, only at release prep.

For EVERY test/debug device + every device under known manual testing, across EVERY transport (USB / wireless ADB / SSH / serial / network introspection API), ALWAYS live-record all analysable data: activities, all logs, perf metrics (CPU/mem/I/O/thermal/load), every sink-side report (§11.4.13), any live-changeable param. (1) **Side-effect-free** — non-invasive read-only probes, bounded sampling + write-volume, observer-effect budget; a perturbing recorder = §11.4.128 violation. (2) **Background + parallel + subagent-driven** (§11.4.103 + §11.4.70) — never blocks main stream. (3) **Token-conscious record-now-analyse-later** — raw NOT processed without need; only analyse-trigger = release-tag prep (§11.4.40/§11.4.42) OR explicit operator ask. (4) **Raw git-ignored (w/ §11.4.77 regen declaration) + code-intelligence-excluded (§11.4.78/79)** — only curated evidence committed, only at release prep under docs/qa/<run-id>/ (§11.4.83). (5) **Layout** <recording-root>/YYYY-MM-DD/<combined main+submodules hash>/<DEVICE>_<SERIAL>/recording_NNN/<files>. (6) **Anti-bluff** — recorder-running-but-no-growing-corpus = §11.4 bluff; curated findings trace to real raw paths; recorder health = captured evidence (§11.4.5/69). Composes

§11.4.2/.5/.13/.69/.40/.42/.70/.77/.78/.79/.83/.103/.119.
Classification: universal (§11.4.17). Gate CM-COVENANT-114-128-PROPAGATION (literal 11.4.128) + rec. gate CM-DEVICE-RECORDING-ALWAYS-ON + paired §1.1 mutation.

Canonical authority: Constitution.md §11.4.128. Release blocker. No --skip-recording / --record-without-layout / --commit-raw-corpus / --index-raw-corpus / --analyse-corpus-always / --invasive-probe-OK escape.

§11.4.129 — Huge-blocker release protocol (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): on a huge blocker during release validation: STOP all testing → fix ALL discovered issues → process all recorded data from last session → land rock-solid fixes → author NEW validation+verification tests of ALL supported test types → rebuild → reflash → RESTART full validation+verification of every fix/change from the last release tag to now — both devices in parallel, recorded, real physical proofs, no bluff.

On a HUGE BLOCKER (release-blocking-severity: core capability broken / regression invalidating the cycle / blast radius reaching the batch's other fixes), in order, NO shortcut: (1) STOP all testing every device (§11.4.4 STOP at release granularity). (2) Fix ALL discovered issues (not just the blocker) — root-cause §11.4.102 + isolate against last known-good tag §11.4.114. (3) Process all recorded data from last session (the §11.4.128(3) release-prep analyse-trigger). (4) Land rock-solid fixes §11.4.123 + §11.4.43/.115 + §11.4.9. (5) Author NEW tests of ALL supported types §11.4.27 + §11.4.85, anti-bluff + paired §1.1 mutation. (6) Rebuild (full, not module-only) + reflash CLEAN target §11.4.108. (7) RESTART full validation from last tag to now §11.4.40 — RESTART not resume — both/all devices parallel §11.4.103/.119, recorded §11.4.128, real physical proofs §11.4.5/.69/.107, no bluff. BINDS the existing release anchors for the huge-blocker case (STOP→fix-all→process-recordings→new-tests-all-types→rebuild→reflash→full-restart + restart-not-resume), citing not duplicating. Composes §11.4.4/.40/.42/.9/.27/.85/.102/.108/.114/.115/.123/.128/.103/.119. Classification: universal (§11.4.17). Gate CM-COVENANT-114-129-PROPAGATION (literal 11.4.129) + rec. gate CM-HUGE-BLOCKER-FULL-RESTART + paired §1.1 mutation.

Canonical authority: Constitution.md §11.4.129. Release blocker. No --resume-after-blocker / --spot-validate-after-fix / --skip-recording-analysis / --skip-new-tests / --module-only-after-blocker / --single-device-restart escape.

§11.4.130 — Post-remediation validate-the-fix-FIRST-after-redeploy (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): when a blocker found during release validation is fixed and a new artifact (rebuild / new flashing image / redeploy) is produced + the target reflashed, FIRST re-test the SPECIFIC last-failing features + validate the just-incorporated fixes BEFORE the broader / full validation.

When a blocker / critical failure found during release validation is FIXED and a new artifact is produced + the target reflashed / re-distributed / updated, in order, NO shortcut: (1) re-test the SPECIFIC last-failing features FIRST (targeted guard tests for exactly the defects this fix addressed) BEFORE any broader / full-suite validation; (2) validate the just-incorporated fixes with real captured evidence — the §11.4.115 RED test flips GREEN at RED_MODE=0 on the new artifact AND the §11.4.108 runtime-signature verifies on the CLEAN target the redeploy produced (metadata-only/config-only/absence-of-error/grep-without-runtime PASS forbidden §11.4/.1; proof §11.4.5/.69/.107/.123); (3) ONLY after the targeted fix is CONFIRMED working proceed to the §11.4.40 full retest from last tag to now. Rationale: a first fix attempt may not work / be incomplete / regress again under the new artifact — confirming FIRST catches fix-did-not-take immediately instead of hours later at the END of a full cycle (then restarting §11.4.129); cheap-confirmation-first is §11.4.82 applied to the post-blocker reflash. This is §11.4.46 specialised for the post-blocker-reflash case + the targeted-confirmation phase that GATES §11.4.129's step-7 full-restart. Honest boundary §11.4.6: "probably took" ≠ "took"; a still-FAILing targeted re-test re-enters the §11.4.114/.115 isolate→RED→fix loop, never proceeds to the full cycle on a still-broken fix. Composes §11.4.4/.40/.46/.108/.114/.115/.123/.129/.82. Classification: universal (§11.4.17). Gate CM-COVENANT-114-130-PROPAGATION (literal 11.4.130) + rec. gate CM-FIX-FIRST-AFTER-REDEPLOY + paired §1.1 mutation.

Canonical authority: [Constitution.md](#) §11.4.130. Release blocker. No --skip-targeted-retest / --full-cycle-first / --assume-fix-took / --validate-fix-at-end / --skip-red-green-flip-on-new-artifact escape.